MEG-001 — Go Engineering Standards

External publication draft

MEG-001 — Go Engineering Standards

> *Good Go code is not measured by cleverness. It is measured by how quickly another engineer can understand, trust and safely change it.*

Purpose

The Mosaic Engineering Guidelines (MEG) establish the engineering standards for every Go codebase within the Mosaic ecosystem.

Where the Mosaic Design Language (MDL) defines how the platform thinks, and the Mosaic Design Specifications (MDS) define how the platform presents itself, the MEG defines how Mosaic software is engineered.

It provides a single source of truth for:

- Architectural decisions
- Engineering philosophy
- Code organisation
- Design patterns
- Package structure
- Concurrency
- Error handling
- Testing
- Performance
- Code quality
- Long-term maintainability

Unlike many language style guides, MEG-001 is **not** a collection of syntax rules.

It defines the engineering philosophy through which every future Mosaic service, application and SDK should be implemented.

Relationship to Mosaic Architecture

```
MDL
↓
Product Philosophy
↓
MDS
↓
Design System
↓
MEG
↓
Engineering Standards
↓
Implementation
↓
Runtime Behaviour
```

Engineering standards are the bridge between architecture and implementation.

Every line of Go code written for Mosaic should be explainable by this specification.

Scope

This specification defines:

- Engineering philosophy
- Idiomatic Go practices
- Project structure
- Package boundaries
- Dependency management
- Abstraction strategy
- Interface design
- Composition
- Error handling
- Concurrency
- Testing philosophy
- Performance principles
- Design patterns

- Anti-patterns
- Code review standards
- Contributor expectations

This specification intentionally does **not** define:

- Product architecture
- Business logic
- API contracts
- Database schemas
- User interface design
- Design system implementation

Those are defined by the MDL, MDS and future architecture specifications.

Core Question

MEG-001 exists to answer one question.

> ***How should Go software be engineered within the Mosaic ecosystem?***

Engineering Statement

Within Mosaic:

> ***Code exists to communicate intent before it executes behaviour.***

Readable software is maintainable software.

Simple software is reliable software.

Architecture should make the correct implementation the easiest implementation.

Engineering Hierarchy

The Mosaic Engineering Standards intentionally separate engineering concerns into conceptual layers.

```
Engineering Philosophy

↓

Architectural Principles

↓

Project Structure

↓

Package Design

↓
```

```
Implementation Patterns
↓
Runtime Behaviour
↓
Operational Excellence
```

Each layer has exactly one responsibility.

Future chapters define every layer in detail.

Expected Outcome

After reading MEG-001 contributors should understand:

- how Mosaic Go projects are organised
- why Go differs from object-oriented languages
- when abstraction is appropriate
- how packages should evolve
- how concurrency should be applied
- how engineering decisions are evaluated
- how code reviews are performed
- how future engineering standards should evolve

without discussing specific business features or product implementations.

Repository Structure

```
engineering/

■■■ meg/

    ■■■ MEG-001 Go Engineering Standards/

        README.md

        00-document-control.md

        01-engineering-philosophy.md

        02-thinking-in-go.md

        03-project-structure.md

        04-package-design.md

        05-dependency-management.md

        06-interfaces-and-abstraction.md

        07-composition-and-polymorphism.md

        08-error-handling.md

        09-context-and-cancellation.md

        10-concurrency.md

        11-testing.md

        12-performance.md

        13-design-patterns.md

        14-anti-patterns.md

        15-code-review-standards.md

        16-boy-scout-rule.md

        17-adrs.md

        18-contributor-guidance.md

        glossary.md

        references.md
```

Dependencies

Required reading:

- MDL-001 Vision
- MDL-002 Principles
- MDL-003 Mental Model
- MDL-004 Interaction Model
- MDL-005 Composition Model

Recommended references:

- Effective Go
- Go Code Review Comments
- Go Style Guide

Design Goals

The standards defined by MEG-001 are intended to produce software that is:

- Idiomatic
- Predictable
- Maintainable
- Observable
- Testable
- Performant
- Evolvable
- Easy to review
- Easy to refactor

Optimisation should never come at the expense of clarity without measurable justification.

Review Status

Status

Draft

Owner

Lead Software Architect

Next File

`00-document-control.md`

Document Control

Document Information

Field	Value
Document ID	MEG-001
Title	Go Engineering Standards
File	00-document-control.md
Status	Draft
Version	0.1
Owner	Lead Software Architect
Classification	Internal Architecture Specification

Purpose

This document establishes the governance, authority and lifecycle of the Mosaic Engineering Guidelines (MEG).

It defines how engineering standards are created, reviewed and maintained throughout the Mosaic ecosystem.

Unlike implementation documentation, MEG specifications are considered **normative architectural standards**.

Where guidance within this specification conflicts with implementation, the discrepancy must be resolved rather than ignored.

Authority

MEG specifications define the canonical engineering standards for every Go codebase within Mosaic.

This includes, but is not limited to:

- Backend services
- APIs
- Workers
- Background jobs
- SDKs
- CLI applications
- Infrastructure tooling
- Shared libraries

Individual repositories may introduce additional standards provided they do not conflict with this specification.

Normative Language

Unless explicitly stated otherwise, the keywords below are interpreted using the definitions established by RFC 2119.

Keyword	Meaning
MUST	Mandatory requirement.
MUST NOT	Behaviour that is prohibited.
SHOULD	Strong recommendation. Deviation requires clear justification.
SHOULD NOT	Generally discouraged. Exceptions should be documented.
MAY	Optional behaviour determined by engineering judgement.

Examples and code samples contained within this specification are **informative**, unless explicitly stated otherwise. This mirrors common standards-writing practice where examples explain intent rather than introduce mandatory behaviour.
[oai_citation:0‡W3C](https://w3c.github.io/manual-of-style/?utm_source=chatgpt.com)

Document Lifecycle

Every MEG specification progresses through the following lifecycle.

```
Draft

↓

Review

↓

Accepted

↓

Implemented

↓

Maintained

↓

Superseded (optional)
```

Engineering standards are expected to evolve alongside the Mosaic platform.

The handbook is therefore considered a living specification.

Change Management

Changes to accepted engineering standards SHOULD be made through an Architectural Decision Record (ADR).

Minor improvements may be made directly when they:

- improve clarity

- correct mistakes
- expand examples
- add references
- improve terminology

Changes that alter engineering philosophy, architectural direction or implementation expectations MUST be accompanied by an ADR.

Compliance

All Go repositories within the Mosaic organisation SHOULD comply with this specification.

Where deviation is necessary, the repository SHOULD document:

- the reason
- the expected impact
- why the standard cannot reasonably be followed

Temporary deviations should eventually be removed.

Permanent deviations should result in an update to the handbook if they represent a better engineering approach.

Engineering Philosophy

The purpose of these standards is not to maximise theoretical purity.

The purpose is to produce software that is:

- understandable
- maintainable
- predictable
- testable
- observable
- performant
- enjoyable to work on

Engineering judgement always takes precedence over blindly following rules.

A rule that consistently makes software worse should be challenged through architecture review rather than ignored.

External References

The MEG builds upon established Go guidance rather than replacing it.

Primary references include:

- Effective Go
- Go Code Review Comments
- Go Documentation Guidelines

Where Mosaic introduces additional conventions, those conventions exist to improve consistency across the project rather than redefine the Go language itself. The Go project itself describes the Code Review Comments as a supplement to *Effective Go*, not a complete style guide, which is the philosophy adopted here. [oai_citation:1‡Google Source](https://go.googlesource.com/wiki/%2B/6fe9f52ac7c4d92cb8fc878d8dee1bda0c63c8a5/CodeReviewComments.md?utm_source=chatgpt.com)

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`README.md`

Next File

`01-engineering-philosophy.md`

Engineering Philosophy

> *Architecture determines how software evolves. Engineering philosophy determines why it evolves that way.*

Purpose

Every engineering decision made within Mosaic should be guided by a small set of enduring principles rather than personal preference.

Programming languages evolve.

Frameworks come and go.

Libraries are replaced.

Good engineering principles remain remarkably stable.

This document establishes the philosophical foundation upon which every future Go engineering standard is built.

Philosophy Statement

Within Mosaic:

> ***Software is engineered for the next engineer, not the current one.***

Every implementation should optimise for:

- Understanding
- Maintainability
- Predictability
- Correctness
- Observability
- Evolvability

The primary consumer of source code is not the compiler.

It is another engineer.

Core Principles

1. Simplicity Over Cleverness

> ***"Simple things should be simple."***

The simplest solution that correctly solves the problem is almost always preferred.

Complexity should only be introduced when it provides measurable value.

Avoid engineering solutions that require significant explanation to understand.

If an implementation appears "clever", it should be assumed to require further simplification.

Why

Simple systems are easier to:

- review
- debug
- optimise
- extend
- replace

Complexity accumulates interest.

2. Explicit Over Implicit

Hidden behaviour creates hidden bugs.

Software should communicate what it is doing directly.

Avoid:

- hidden state
- implicit side effects
- surprising control flow
- reflection where unnecessary
- unnecessary indirection

A reader should rarely need to leave the current function to understand what is happening.

Example

Prefer:

```
[go]
service.Start(ctx)
```

Over:

```
[go]
service.Execute()
```

where `Execute()` internally creates goroutines, opens database connections and ignores cancellation.

Behaviour should be visible.

3. Readability Is A Feature

Readable code is easier to verify.

Easier to test.

Easier to optimise.

Easier to replace.

Readable software therefore has a lower long-term cost.

A function should communicate intent before implementation.

If comments are required to explain *what* code is doing, the implementation should usually be rewritten.

Comments should explain **why**, not **what**.

4. Design For Change

Requirements will change.

Architecture should make those changes inexpensive.

Good engineering reduces the cost of future change.

This generally means:

- small packages
- clear boundaries
- loose coupling
- explicit dependencies
- cohesive responsibilities

Software should be difficult to break accidentally.

5. Favour Composition

Mosaic adopts composition as its primary mechanism for building complex systems.

Large inheritance hierarchies are intentionally avoided.

Instead:

Small components collaborate.

Small packages compose.

Small interfaces connect systems.

Complex behaviour emerges from combining simple responsibilities.

Later chapters define how composition should be applied throughout Go projects.

6. Make The Correct Thing Easy

Good architecture encourages correct behaviour.

Poor architecture relies upon discipline.

Whenever possible:

- invalid states should be impossible
- dangerous operations should be difficult
- correct usage should be obvious

Good APIs reduce mistakes by design.

7. Optimise Last

Performance matters.

Premature optimisation does not.

Optimisation should be driven by:

- profiling
- measurement
- production evidence

Never by assumption.

Simple code that is slightly slower is usually preferable to complicated code that is theoretically faster.

When optimisation becomes necessary, it should be isolated and well documented.

This reflects long-standing Go guidance to measure first and optimise second.
([go.dev](https://go.dev/blog/profiling-go-programs))

8. Consistency Over Preference

Every engineer has preferences.

A shared codebase cannot.

Consistency reduces cognitive load.

The best engineering standard is often not the theoretically perfect one, but the one applied consistently across the entire platform.

Future contributors should spend their time understanding business logic rather than adapting to different coding styles.

9. The Compiler Is A Design Tool

Go's type system exists to eliminate categories of bugs before software executes.

Types should communicate intent.

Interfaces should model behaviour.

The compiler should detect mistakes wherever practical.

Whenever a runtime validation can reasonably become a compile-time guarantee, the compile-time solution should be preferred.

10. Engineering Is Communication

Every engineering artefact communicates.

Source code communicates.

Tests communicate.

Documentation communicates.

Naming communicates.

Architecture communicates.

The goal is not merely to produce software that functions.

The goal is to produce software whose intent remains obvious years after it was written.

Decision Framework

When evaluating multiple implementations, engineers SHOULD ask the following questions.

1. Which implementation is easiest to understand? 2. Which implementation is easiest to test? 3. Which implementation introduces the least coupling? 4. Which implementation is easiest to change? 5. Which implementation communicates intent most clearly? 6. Which implementation aligns with established Go conventions? 7. Which implementation best supports the long-term health of the codebase?

The implementation that scores highest overall should generally be preferred.

Engineering Priorities

When trade-offs exist, Mosaic evaluates engineering decisions using the following priority order.

```
Correctness

↓

Maintainability

↓

Readability

↓

Testability

↓

Observability

↓

Performance

↓

Convenience
```

This ordering intentionally places long-term maintainability above short-term development speed.

Convenience should never justify unnecessary complexity.

Boy Scout Rule

Every engineer shares responsibility for improving the codebase.

Whenever modifying existing code:

- improve naming where appropriate
- simplify unnecessary complexity
- remove dead code
- improve documentation
- increase test coverage
- reduce duplication

Small improvements made consistently produce significant long-term gains.

Large refactors are not required.

Continuous improvement is.

Philosophy Summary

The engineering philosophy of Mosaic can be summarised as follows.

> *Build software that another engineer can confidently change without fear.*

Every future engineering standard defined within the MEG should reinforce that objective.

If a rule makes software harder to understand, harder to test or harder to evolve, it should be challenged.

Engineering standards exist to improve software.

Never to burden it.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`00-document-control.md`

Next File

`02-thinking-in-go.md`

Thinking in Go

> *Go is not a simpler Java. It is a language built upon a fundamentally different philosophy.*

Purpose

Many engineers arriving at Go have backgrounds in object-oriented languages such as Java, C# or C++.

While experience from those languages remains valuable, directly translating object-oriented design patterns into Go often produces software that feels unnecessarily complex, difficult to maintain and distinctly non-idiomatic.

This document establishes the mental model required to engineer software that embraces Go's strengths rather than fighting against them.

Why This Matters

Most poorly designed Go software is not written by inexperienced programmers.

It is written by experienced programmers attempting to solve Go problems using the tools and philosophies of another language.

Go intentionally omits many language features common elsewhere.

This is not because they are impossible.

It is because the language encourages solving problems differently.

Learning Go therefore requires more than learning syntax.

It requires changing how software is designed.

The Go team explicitly cautions against translating Java or C++ designs directly, encouraging developers to think in Go's own idioms instead.
[oai_citation:0‡Go](https://go.dev/doc/effective_go?v=1&utm_source=chatgpt.com)

The Fundamental Shift

Most object-oriented languages begin with types.

Go begins with behaviour.

Traditional OO thinking often follows:

```
Object
↓
Inheritance
↓
Framework
↓
```

```
Application
```

Go thinking instead follows:

```
Problem

↓

Behaviour

↓

Composition

↓

Application
```

The focus shifts from building class hierarchies to composing small behaviours into larger systems.

Java Thinking vs Go Thinking

Object-Oriented Thinking	Go Thinking
Everything is an object	Everything is a value
Build inheritance trees	Compose small components
Start with abstractions	Start with concrete implementations
Design interfaces first	Discover interfaces when needed
Frameworks organise applications	Packages organise applications
Configuration drives behaviour	Code drives behaviour
Exceptions	Explicit error values
Dependency Injection Containers	Constructor injection
Large service hierarchies	Small cohesive packages

Neither philosophy is universally better.

They optimise for different goals.

Mosaic adopts the Go approach because it produces software that is easier to understand, easier to refactor and easier to operate.

Concrete Before Abstract

One of the most common mistakes made by developers new to Go is introducing abstraction before it has demonstrated value.

For example:

```
UserService

↓

UserServiceImpl

↓
```

`DefaultUserService`

↓

`BaseUserService`

↓

`AbstractUserService`

This hierarchy communicates very little.

Instead, Go encourages:

`UserService`

If only one implementation exists, one type is sufficient.

Additional abstractions should only appear when multiple implementations genuinely require them.

Behaviour Over Hierarchy

Go does not organise software through inheritance.

Instead, software is built by combining behaviours.

Rather than asking:

> *"What does this type inherit?"*

Ask:

> *"What behaviour does this type provide?"*

This small change dramatically influences software architecture.

Systems become collections of collaborating components rather than deeply nested inheritance trees.

Interfaces Are Discovered

In many object-oriented languages, interfaces are designed before implementations.

Within Mosaic, the opposite approach is preferred.

Implement the concrete type first.

Allow the interface to emerge naturally from actual usage.

Interfaces should exist because multiple consumers require a common behaviour.

Not because every service "might one day" need another implementation.

This aligns with common Go guidance that interfaces generally belong near the consumer and should not be created speculatively.
[oai_citation:1‡GitHub](https://github.com/pthethanh/effective-go?utm_source=chatgpt.com)

Fewer Layers

Enterprise software often accumulates unnecessary architectural layers.

```
Controller
↓
Service
↓
Manager
↓
Provider
↓
Repository
↓
DAO
↓
Database
```

Each layer introduces:

- indirection
- coupling
- maintenance cost
- cognitive overhead

Go generally favours fewer, more meaningful boundaries.

Every layer should exist because it owns a distinct responsibility.

Not because another project used the same architecture.

Packages Replace Frameworks

Large object-oriented applications frequently rely on frameworks to define application structure.

Go instead relies upon packages.

Packages define:

- ownership
- responsibility
- visibility
- dependency direction

A well-designed package communicates more architectural intent than dozens of annotations or configuration files.

Later chapters establish Mosaic's package organisation standards.

Explicit Dependencies

Dependencies should always be visible.

Poor:

```
Global state

↓

Hidden singleton

↓

Reflection

↓

Runtime discovery
```

Preferred:

```
main()

↓

Construct dependencies

↓

Inject explicitly

↓

Application starts
```

Reading `main.go` should explain how the application is assembled.

Nothing important should happen "behind the scenes."

Errors Are Expected

Many languages treat errors as exceptional.

Go treats them as ordinary outcomes.

This changes software design considerably.

Instead of assuming success and catching failures later, Go encourages engineers to acknowledge failure immediately.

Error handling therefore becomes part of the normal control flow rather than a separate mechanism.

Future chapters establish Mosaic's error handling philosophy.

Small Things Build Large Systems

Large systems should emerge from combining many small pieces.

Examples include:

- small packages
- small interfaces
- small structs
- small functions
- focused responsibilities

Large components should be considered a design smell until proven necessary.

Engineering Through Constraints

Some developers initially perceive Go as "missing features."

In reality, Go intentionally limits certain language capabilities.

Examples include:

- no inheritance
- no implicit constructors
- no exceptions
- limited metaprogramming
- minimal reflection
- no annotation system

These constraints encourage simpler software.

Rather than asking:

> *"Why doesn't Go allow this?"*

Ask:

> *"What design is Go encouraging instead?"*

Very often, the simpler solution proves easier to maintain.

Mosaic Philosophy

Within Mosaic, engineers SHOULD continuously ask:

- Can this become simpler?
- Is this abstraction necessary?
- Would another engineer immediately understand this?
- Does this follow established Go conventions?
- Am I solving today's problem or an imaginary future problem?

If uncertainty exists, the simpler solution should generally be preferred.

Complexity should justify itself.

Simplicity does not.

Key Takeaways

Thinking in Go means accepting several fundamental ideas.

- Concrete types come before abstractions.
- Behaviour is more important than hierarchy.
- Composition replaces inheritance.
- Packages replace frameworks.
- Explicit dependencies replace runtime magic.
- Simplicity is a competitive advantage.
- Readability is an engineering feature.
- Good architecture removes unnecessary decisions.

These ideas form the foundation upon which every subsequent engineering standard within the MEG is built.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`01-engineering-philosophy.md`

Next File

`03-project-structure.md`

Project Structure

> *A project's directory structure is its first piece of documentation. Before a single line of code is read, contributors should understand where responsibility lives.*

Purpose

A consistent project structure allows engineers to navigate unfamiliar codebases quickly and confidently.

Within Mosaic, directories are organised around **architectural responsibility**, not personal preference.

Every package should have a clear owner, a clear purpose and a clearly defined dependency direction.

The structure should communicate the architecture.

It should never hide it.

Philosophy

Project structure exists to reduce cognitive load.

When engineers know where something belongs, they spend less time searching and more time solving problems.

A well-structured repository should answer questions such as:

- Where does business logic live?
- Where are HTTP handlers implemented?
- Where should database code exist?
- Where are background workers located?
- Where are events defined?
- Where should new features be added?

Without opening a single source file.

Design Principles

The Mosaic project structure is built upon the following principles.

Feature Ownership

Packages own business capabilities.

Examples:

```
library

metadata

playback

authentication
```

rather than technical concepts such as:

```
helpers
```

```
common
```

```
misc
```

Feature-oriented organisation improves discoverability and reduces accidental coupling.

Single Responsibility

Every package SHOULD have one reason to change.

If a package contains unrelated concepts, it should be split.

Large "catch-all" packages inevitably become architectural bottlenecks.

Dependency Direction

Dependencies always flow inward.

```
Transport
```

```
↓
```

```
Application
```

```
↓
```

```
Domain
```

```
↓
```

```
Infrastructure
```

Lower layers MUST NOT depend upon higher layers.

For example:

```
HTTP Handler
```

```
↓
```

```
Service
```

```
↓
```

```
Repository
```

```
↓
```

```
Database
```

The repository must never import an HTTP package.

Likewise, domain logic must never depend on transport concerns.

Encapsulation

Implementation details should remain private.

Packages should expose only what other packages require.

Everything else should remain unexported.

Go's `internal` mechanism exists specifically to enforce package boundaries at compile time and is encouraged for application implementation details. ([go.dev](https://go.dev/doc/modules/layout))

Standard Repository Layout

Every Mosaic service SHOULD follow the same high-level structure.

```
cmd/

internal/

pkg/

api/

configs/

scripts/

docs/

test/
```

Not every repository will require every directory.

Unused directories SHOULD NOT be created simply to satisfy the standard.

Structure should emerge from actual requirements.

Directory Responsibilities

cmd/

Contains application entry points.

Examples:

```
cmd/server

cmd/migrate

cmd/worker
```

Each directory represents an independently executable application.

Business logic MUST NOT be implemented here.

Responsibilities include:

- dependency construction
- configuration loading
- application startup
- graceful shutdown

Nothing more.

internal/

Contains private application code.

Everything inside `internal` is considered implementation detail.

Typical contents include:

```
internal/

    app/

    domain/

    transport/

    infrastructure/

    scheduler/

    events/

    middleware/
```

Most Mosaic code will exist here.

pkg/

Contains reusable packages intended for external consumption.

This directory SHOULD be used sparingly.

Code belongs in `pkg` only if another repository is expected to import it.

Moving code into `pkg` should be considered an architectural commitment.

api/

Contains externally published API contracts.

Examples include:

- OpenAPI specifications
- Protobuf definitions
- JSON schemas

Generated code SHOULD NOT be manually modified.

configs/

Stores example configuration files.

Configuration templates belong here.

Actual secrets MUST NOT.

docs/

Repository-specific documentation.

Architecture specifications belong within the architecture repository rather than individual services whenever practical.

scripts/

Automation utilities.

Examples:

- build scripts
- release scripts
- migration helpers
- local development tooling

Scripts SHOULD remain idempotent whenever possible.

test/

Cross-package integration tests.

Unit tests SHOULD remain beside the package they exercise.

Large end-to-end scenarios belong here.

Package Organisation

Within `internal`, packages SHOULD be organised around business capabilities.

Preferred:

```
internal/
    library/
    metadata/
    playback/
    users/
    authentication/
```

Avoid:

```
internal/
    services/
    managers/
    implementations/
    interfaces/
```

The former communicates purpose.

The latter communicates implementation mechanics.

Purpose should always win.

Naming Conventions

Package names MUST:

- be singular where practical
- use lowercase
- avoid underscores
- avoid abbreviations unless universally recognised
- describe responsibility

Good examples:

```
library
```

```
metadata
```

```
playback
```

Poor examples:

```
helpers
```

```
common
```

```
shared
```

```
utils
```

These names communicate ownership poorly and tend to become dumping grounds for unrelated code.

Package Size

Large packages should be viewed with suspicion.

Signs a package has grown too large include:

- hundreds of exported identifiers
- multiple unrelated responsibilities
- frequent merge conflicts
- unclear ownership
- circular dependencies beginning to emerge

When this occurs, responsibilities SHOULD be extracted into cohesive packages.

Internal Before Shared

Engineers frequently overestimate the likelihood that code will become reusable.

Within Mosaic:

Code SHOULD begin inside `internal`.

Only after multiple repositories demonstrate a genuine need should it be promoted into `pkg`.

Sharing code prematurely often creates tighter coupling than duplication.

Example Structure

A typical Mosaic service may resemble:

```
cmd/
    mosaic-server/
internal/
    app/
    authentication/
    events/
    library/
    metadata/
    playback/
    scheduler/
    transport/
        http/
        websocket/
    infrastructure/
        postgres/
        duckdb/
        blob/
pkg/
api/
configs/
docs/
scripts/
test/
```

This structure balances discoverability, encapsulation and future growth while remaining familiar to experienced Go developers.

Anti-Patterns

The following package names SHOULD NOT exist without exceptional justification.

`utils`

`common`

`base`

`shared`

`misc`

`manager`

`helper`

These packages usually indicate responsibilities have not yet been properly identified.

If a function belongs in `utils`, it probably belongs somewhere else.

Summary

Project structure should communicate architecture.

Every directory should answer:

> ***What responsibility does this own?***

If that question cannot be answered clearly, the structure should be reconsidered.

Good structure enables good engineering.

Poor structure amplifies technical debt long before implementation quality begins to decline.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`02-thinking-in-go.md`

Next File

`04-package-design.md`

Package Design

> *The package is the fundamental unit of architecture in Go. Good packages make good software. Poor packages make every line of code harder to understand.*

Purpose

Packages are the primary mechanism by which Go organises software.

Unlike many object-oriented languages, where classes represent the fundamental organisational unit, Go places architectural responsibility at the package level.

For this reason, package design is one of the most important architectural decisions made within a Go codebase.

This document defines how packages should be designed throughout the Mosaic ecosystem.

Why Packages Matter

Every engineer interacting with a Go project spends most of their time moving between packages.

Good packages answer three questions immediately.

> *What responsibility does this package own?*
>
> *What does this package expose?*
>
> *What should never belong here?*

When those questions are difficult to answer, the package has usually become too broad.

Philosophy

Within Mosaic:

> ***Packages own responsibilities. Types merely implement them.***

Packages are architectural boundaries.

Types are implementation details.

A package should communicate purpose before implementation.

The Responsibility Rule

Every package MUST own exactly one responsibility.

Examples include:

```
library
```

```
metadata
```

```
playback
```

```
authentication
```

A package should never become a collection of unrelated utilities simply because they happen to be used together.

Cohesion

Everything inside a package should naturally belong together.

A useful question is:

> *"Would removing this file make the package easier to explain?"*

If the answer is yes, the file probably belongs elsewhere.

Packages should feel cohesive.

Not convenient.

Coupling

Packages SHOULD minimise knowledge of one another.

The fewer packages required to understand a feature, the easier the software becomes to maintain.

Signs of excessive coupling include:

- frequent circular dependency attempts
- packages importing one another indirectly
- unrelated changes requiring edits across multiple packages
- large dependency graphs

Good package boundaries reduce these problems naturally.

Package Size

Package size should emerge naturally.

Neither extremely small nor extremely large packages are desirable.

A package containing a single type is not automatically better.

Likewise, a package containing fifty unrelated types is almost certainly worse.

Engineers should optimise for conceptual clarity.

Not line counts.

Public API

Every exported identifier becomes part of the package's public contract.

Exporting something should therefore be considered an architectural decision.

Ask:

> *Does another package genuinely need this?*

If the answer is no, keep it unexported.

Smaller public APIs are easier to evolve.

Package Names

Package names SHOULD be:

- short
- descriptive
- lowercase
- singular where practical
- nouns rather than verbs

Good examples:

```
metadata
```

```
library
```

```
playback
```

```
scheduler
```

Poor examples:

```
metadataservice
```

```
helpers
```

```
common
```

```
misc
```

```
stuff
```

Go package names become part of every import statement.

Short, meaningful names improve readability throughout the codebase. The Go team explicitly recommends short, clear package names and discourages meaningless names such as `util`, `common`, `api`, `types` or `interfaces`. [oai_citation:0‡Go](https://go.dev/blog/package-names?utm_source=chatgpt.com)

Avoid Package Stutter

Identifiers should not repeat the package name.

Poor:

```
[go]
package library

type LibraryService struct{}
```

Usage:

```
[go]
```

```
library.LibraryService
```

Better:

```
[go]
package library

type Service struct{}
```

Usage:

```
[go]
library.Service
```

Or better still, if the type represents the domain:

```
[go]
package library

type Manager struct{}
```

or

```
[go]
package library

type Catalogue struct{}
```

The package already provides context.

Avoid repeating it.

Avoid Generic Packages

The following package names SHOULD NOT exist.

```
util

utils

common

shared

base

types

interfaces

helpers
```

These names communicate implementation convenience rather than architectural responsibility.

Over time they become dumping grounds.

If something belongs in `utils`, its true ownership has probably not yet been identified.

Package Ownership

Every package should have a clear owner.

Ownership answers:

- what belongs here

- what does not belong here
- who is responsible for changes

Example:

```
metadata
```

Owns:

- metadata models
- metadata ingestion
- metadata translation
- metadata providers

It does not own:

- HTTP handlers
- playback
- authentication
- persistence for unrelated domains

Responsibilities should not leak.

Internal Collaboration

Packages should collaborate through exported behaviour.

They should not rely upon one another's implementation details.

Good:

```
metadata.Provider
```

↓

```
library.Service
```

Poor:

```
library
```

↓

```
metadata/internal/parser
```

↓

```
metadata/private/cache
```

If another package requires access to implementation details, the abstraction is probably incorrect.

Package Dependencies

Imports should naturally form a directed graph.

Preferred:

```
transport
↓
application
↓
domain
↓
infrastructure
```

Avoid:

```
library
↓
metadata
↓
library
```

Circular dependencies are prohibited by the Go compiler.

Rather than viewing this as a limitation, Mosaic treats it as an architectural safeguard.

If two packages require one another, they probably represent a single responsibility or require a new shared abstraction.

Package Documentation

Every exported package MUST include a package comment.

The package comment should explain:

- why the package exists
- what responsibility it owns
- what it intentionally does not own

Example:

```
[go]
// Package metadata manages the ingestion, transformation and
// persistence of media metadata from external providers.
package metadata
```

Package comments are part of Go's documentation conventions and are expected for exported packages. [oai_citation:1‡Go](https://go.dev/wiki/CodeReviewComments?utm_source=chatgpt.com)

When To Create A New Package

A new package SHOULD only be created when at least one of the following becomes true.

- A new business capability exists.
- Dependencies need separating.

- Visibility needs restricting.
- Ownership has changed.
- Reuse naturally emerges.

A package SHOULD NOT be created merely because:

- a file is becoming large
- another project used the same structure
- "everything should have its own package"

Packages should emerge from architecture.

Not aesthetics.

Mosaic Package Principles

Every package within Mosaic should satisfy the following checklist.

- Own one responsibility.
- Have one reason to change.
- Expose the smallest possible API.
- Hide implementation details.
- Minimise dependencies.
- Avoid circular knowledge.
- Communicate purpose through its name.
- Be easy to explain in one sentence.

If these conditions are not met, the package should be reconsidered.

Summary

Good packages make software understandable.

They reduce coupling.

They improve discoverability.

They encourage clear ownership.

Within Mosaic, packages are considered architectural boundaries rather than folders.

They should therefore be designed with the same care as public APIs.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`03-project-structure.md`

Next File

`05-dependency-management.md`

Dependency Management

> *Dependencies should be visible, intentional and easy to reason about. If an engineer cannot understand how an application is assembled by reading* `main.go`*, the architecture is hiding too much.*

Purpose

Every application is a graph of dependencies.

Databases depend on configuration.

Repositories depend on databases.

Services depend on repositories.

Handlers depend on services.

Understanding how those dependencies are created is fundamental to understanding the application itself.

This document defines how dependencies should be managed throughout the Mosaic ecosystem.

Philosophy

Within Mosaic:

> ***Dependencies should be constructed explicitly and passed deliberately.***

Nothing should appear "by magic".

An engineer reading the application's entry point should be able to understand exactly:

- what is created
- in what order
- why it exists
- who owns it
- where it is passed

The application's construction should be obvious.

Dependency Graph

Every Mosaic application follows the same construction flow.

```
Configuration
↓
Infrastructure
↓
Repositories
↓
```

```
Services
↓
Transport
↓
Application
```

Dependencies should always move in one direction.

Construction begins with foundational components and progresses towards the application's external interfaces.

Composition Root

Every executable application MUST have a single composition root.

Typically this is:

```
cmd/server/main.go
```

or

```
cmd/worker/main.go
```

The composition root is responsible for:

- loading configuration
- creating infrastructure
- constructing dependencies
- wiring services together
- starting the application
- graceful shutdown

It should **not** contain business logic.

Constructor Injection

Dependencies MUST be supplied through constructors.

Example:

```
[go]
repo := metadata.NewRepository(db)

service := metadata.NewService(repo)

handler := http.NewMetadataHandler(service)
```

Dependencies are visible.

Ownership is obvious.

Testing becomes straightforward.

Avoid Global State

Global mutable state SHOULD NOT exist.

Poor:

```
[go]
var Database *sql.DB
```

Every package can now modify shared state.

Dependencies become hidden.

Testing becomes difficult.

Preferred:

```
[go]
func NewRepository(db *sql.DB) *Repository
```

The dependency is explicit.

Every caller understands what the repository requires.

Avoid Service Locators

The following pattern is prohibited.

```
[go]
service := container.Resolve("metadata")
```

Or:

```
[go]
app.Services.Metadata()
```

These approaches hide dependencies behind runtime lookups.

Instead, dependencies should be supplied directly.

Reading a constructor should reveal every dependency required for that component.

Hidden dependencies make software harder to understand and significantly more difficult to test.

Dependency Injection Containers

Mosaic intentionally does **not** use dependency injection containers.

Examples include:

- Spring
- Guice
- Autofac
- Uber Dig
- Facebook Inject

These frameworks solve problems that Go intentionally avoids through explicit construction.

Reflection-based dependency injection introduces:

- hidden behaviour
- runtime failures
- more difficult debugging
- increased cognitive load

Go applications are typically small enough that manual construction remains the simpler solution.

This is the approach encouraged throughout the Go ecosystem.
([go.dev](https://go.dev/doc/effective_go?v=1&utm_source=chatgpt.com))

Constructor Design

Constructors SHOULD:

- validate required dependencies
- initialise internal state
- return fully usable objects

Constructors SHOULD NOT:

- start goroutines
- perform network requests
- open database transactions
- execute business logic
- perform long-running work

Construction should be inexpensive.

Application behaviour begins after construction completes.

Required Dependencies

Required dependencies MUST be constructor parameters.

Example:

```
[go]
func NewService(
    repo Repository,
    publisher events.Publisher,
    logger *slog.Logger,
) *Service
```

If the service cannot function without a dependency, that dependency should not be optional.

Optional Dependencies

Optional behaviour should be configured explicitly.

Preferred:

```
[go]
service := NewService(repo)

service.EnableMetrics(metrics)
```

Or through functional options where the configuration surface becomes sufficiently large.

Later chapters discuss when functional options are appropriate.

Ownership

The creator of an object owns its lifecycle.

Example:

```
main()

↓

database

↓

repository

↓

service

↓

handler
```

The handler should not close the database.

The repository should not stop the HTTP server.

Ownership always remains with the component that created the dependency.

Lifecycle Management

Long-lived dependencies should be created once.

Examples include:

- database pools
- HTTP clients
- loggers
- event buses
- schedulers

Repeated construction increases resource usage unnecessarily.

Where a dependency is intended to be shared, share the instance rather than recreating it.

Interfaces As Dependencies

Depend on behaviour.

Not implementation.

Preferred:

```
[go]
type Repository interface {
    Find(ctx context.Context, id string) (*Media, error)
}
```

The service depends upon behaviour.

Not a concrete database implementation.

However, interfaces should only be introduced when multiple consumers genuinely benefit.

Do not create interfaces "just in case."

The Go community commonly summarises this principle as:

> ***Accept interfaces. Return concrete types.***

Future chapters explore this principle in detail.

Testing

Explicit dependency construction makes testing straightforward.

Example:

```
[go]
repo := NewFakeRepository()

service := NewService(repo)
```

No framework.

No runtime configuration.

No reflection.

Tests remain fast and deterministic.

Dependency Direction

Dependencies should always point towards stable abstractions.

```
HTTP

↓

Application

↓

Domain

↓
```

```
Infrastructure
```

Infrastructure must never depend on transport.

Repositories must never import HTTP packages.

Domain services must never know whether requests originated from:

- HTTP
- CLI
- WebSocket
- Scheduler
- Worker

Business logic should remain transport agnostic.

Example Composition Root

A typical Mosaic service should resemble:

```
[text]
Load configuration

↓

Create logger

↓

Create database pools

↓

Create repositories

↓

Create services

↓

Create HTTP handlers

↓

Register routes

↓

Start HTTP server

↓

Wait for shutdown signal

↓

Gracefully terminate
```

An engineer unfamiliar with the project should understand the application's entire construction by reading this file.

Anti-Patterns

The following practices SHOULD NOT be used.

Hidden Singletons

```
database.Get()
```

Service Locator

```
container.Resolve(...)
```

Runtime Dependency Discovery

```
reflect.New(...)
```

Circular Construction

```
Service A

↓

Service B

↓

Service A
```

Constructors With Side Effects

```
NewService()

↓

Starts goroutines

↓

Creates timers

↓

Calls APIs
```

Construction should never unexpectedly begin application behaviour.

Summary

Dependency management within Mosaic is intentionally simple.

- Construct explicitly.
- Inject directly.
- Avoid global state.
- Avoid runtime magic.

- Keep ownership obvious.
- Make application startup readable.

If an engineer can understand an application's dependency graph by reading `main.go`, the architecture is probably moving in the right direction.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`04-package-design.md`

Next File

`06-interfaces-and-abstraction.md`

Interfaces and Abstraction

> *Abstraction should remove complexity, not introduce it. Every interface should exist because it solves a real problem, not because another language would have required one.*

Purpose

Interfaces are one of Go's most powerful language features.

They are also one of the most commonly misunderstood.

Engineers transitioning from object-oriented languages frequently overuse interfaces, introducing unnecessary abstraction layers that reduce readability without improving flexibility.

This document defines how interfaces and abstractions should be designed throughout the Mosaic ecosystem.

Philosophy

Within Mosaic:

> ***Concrete first. Abstract second. Interface last.***

An interface is not the starting point of a design.

It is the result of discovering common behaviour between independently useful implementations.

Abstraction should emerge naturally.

It should never be assumed.

What Is An Interface?

An interface describes **behaviour**.

It does **not** describe:

- ownership
- implementation
- storage
- inheritance
- lifecycle

For example:

```
[go]
type Publisher interface {
    Publish(ctx context.Context, event Event) error
}
```

The interface communicates one thing.

> *Something can publish events.*

It intentionally says nothing about:

- Kafka
- NATS
- RabbitMQ
- WebSockets
- Memory

That decision belongs to the implementation.

Why Interfaces Exist

Interfaces exist to reduce coupling.

They allow one component to depend upon behaviour rather than implementation.

This enables:

- testing
- substitution
- modularity
- evolution

Interfaces should therefore be introduced only when they reduce coupling.

Adding an interface that nobody needs increases complexity.

Start With Concrete Types

Every implementation should begin as a concrete type.

Example:

```
[go]
type Service struct {
    repo *Repository
}
```

Do not begin here:

```
[go]
type Service interface {
    Create(...)
    Delete(...)
    Update(...)
}
```

The interface has no consumers.

It communicates nothing.

It merely adds another file to maintain.

Concrete implementations are easier to understand.

Interfaces can always be extracted later.

Interfaces Belong To The Consumer

One of the most important Go design principles is:

> ***The consumer owns the interface.***

Example:

```
metadata

↓

library
```

If `library` only needs:

```
[go]
Find(id string)
```

then `library` defines:

```
[go]
type MetadataProvider interface {
    Find(ctx context.Context, id string) (*Media, error)
}
```

The metadata package simply satisfies it.

The metadata package does **not** need to know the interface exists.

This produces smaller, more focused abstractions and aligns with widely accepted Go design guidance. ([go.dev](https://go.dev/doc/effective_go?v=1&utm_source=chatgpt.com))

Accept Interfaces

Functions SHOULD accept interfaces whenever behaviour is required.

Example:

```
[go]
func NewLibraryService(
    repo Repository,
) *Service
```

The service depends on behaviour.

Not implementation.

This allows:

- production repositories
- in-memory repositories
- mock repositories
- future repositories

without changing the service.

Return Concrete Types

Constructors SHOULD return concrete implementations.

Preferred:

```
[go]
func NewService(...) *Service
```

Avoid:

```
[go]
func NewService(...) Service
```

Returning concrete types allows future API evolution.

Callers remain free to define their own interfaces if required.

Returning interfaces unnecessarily restricts future development.

Keep Interfaces Small

Interfaces SHOULD describe one responsibility.

Excellent:

```
[go]
type Reader interface {
    Read([]byte) (int, error)
}
```

Good:

```
[go]
type Publisher interface {
    Publish(context.Context, Event) error
}
```

Poor:

```
[go]
type MediaService interface {

    Find()

    Update()

    Delete()

    Publish()

    Sync()

    Backup()

    Export()

    Import()

    Validate()
}
```

Large interfaces become difficult to implement, difficult to mock and difficult to understand.

Interface Segregation

Consumers should depend only upon behaviour they require.

Instead of:

```
[text]
Repository

↓

Everything
```

Prefer:

```
Finder

Writer

Publisher

Scheduler
```

Each consumer receives only the behaviour it needs.

Avoid "Impl"

Avoid names such as:

```
RepositoryImpl

DefaultRepository

UserServiceImpl
```

These names usually indicate object-oriented thinking rather than Go thinking.

Instead:

```
Repository

Service

Store

Client
```

Implementation details should remain invisible.

Avoid Premature Abstraction

Do not introduce interfaces because:

- "we might have another implementation"
- "Java does this"
- "everything should be testable"

Instead ask:

> *Is there currently more than one consumer?*

Is this coupling causing a problem?
Does an interface simplify the architecture?

If not:

Do not create one.

When To Introduce An Interface

Interfaces become appropriate when:

- multiple packages depend upon the same behaviour
- testing genuinely benefits
- different implementations naturally exist
- infrastructure varies while behaviour remains stable

Examples include:

- storage providers
- event publishers
- authentication providers
- metadata providers
- external APIs

Abstraction Smells

The following usually indicate unnecessary abstraction.

Single Implementation

```
Repository

↓

RepositoryImpl
```

Interface With One User

```
Service

↓

ServiceImpl

↓

One caller
```

Mirror Interfaces

```
Every struct
```

↓

```
Matching interface
```

Generic Interfaces

```
Manager
```

```
Provider
```

```
Processor
```

without clearly defined behaviour.

Empty Interfaces

```
interface{}
```

or

```
any
```

used to avoid designing proper types.

Generics and any should communicate genuine flexibility.

They should never hide uncertainty.

Prefer Behaviour Over Type

Poor:

```
Can this object become another subclass?
```

Better:

```
What behaviour does this consumer require?
```

This small change fundamentally alters software architecture.

Behaviour becomes modular.

Types remain simple.

Mosaic Guidelines

Within Mosaic:

- Interfaces MUST represent behaviour.
- Interfaces SHOULD belong to consumers.
- Constructors SHOULD return concrete types.
- Functions SHOULD accept interfaces where appropriate.

- Interfaces SHOULD remain small.
- Interfaces SHOULD emerge naturally.
- Large "god interfaces" are prohibited.

If an interface cannot be explained in one sentence, it is probably too large.

Summary

Interfaces are among Go's most elegant features.

Used well, they reduce coupling and improve flexibility.

Used poorly, they recreate the complexity of inheritance-based object-oriented systems.

Within Mosaic, abstraction is considered successful when:

- behaviour becomes easier to understand
- coupling decreases
- testing becomes simpler
- implementation details disappear

If abstraction increases complexity instead, it has failed.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`05-dependency-management.md`

Next File

`07-composition-and-polymorphism.md`

Composition and Polymorphism

> *Composition builds systems. Polymorphism enables flexibility. They solve different problems and should never be confused.*

Purpose

Go deliberately avoids classical inheritance.

Instead, it separates three concepts that inheritance traditionally combines:

- Code reuse
- Behavioural polymorphism
- Object hierarchy

Within Mosaic, these concerns are treated independently.

Composition is used for building systems.

Interfaces are used for polymorphism.

Embedding is used sparingly as a convenience.

This separation produces software that is easier to understand, easier to evolve and significantly less coupled than inheritance-based designs. Go's own documentation describes embedding as a mechanism for code reuse rather than subclassing. [oai_citation:0‡Go](https://go.dev/doc/effective_go?utm_source=chatgpt.com)

Philosophy

Within Mosaic:

> ***Compose behaviour. Never inherit implementation.***

Complex systems should emerge from combining small, focused components.

No component should exist merely because another component "extends" it.

Instead, components collaborate.

Three Different Concepts

One of the biggest misconceptions among engineers transitioning from Java is assuming composition, embedding and polymorphism are interchangeable.

They are not.

Each exists for a different purpose.

Concept	Purpose
Composition	Build larger behaviour from smaller components
Embedding	Reduce delegation boilerplate

Concept	Purpose
Interfaces	Enable behavioural polymorphism

Keeping these concepts separate leads to significantly simpler software.

Composition

Composition is the primary architectural mechanism within Mosaic.

Example:

```
[go]
type LibraryService struct {
    metadata MetadataProvider
    events   Publisher
    cache     Cache
}
```

The service does not inherit functionality.

Instead, it collaborates with other components.

Each dependency owns its own responsibility.

The resulting service becomes easier to:

- understand
- replace
- test
- extend

Think In Capabilities

Instead of asking:

> *What should this object inherit?*

Ask:

> *What capabilities does this component require?*

Example.

```
Playback Service

↓

Metadata Provider

↓

Transcoder

↓

Progress Store

↓
```

```
Event Publisher
```

Each capability remains independently replaceable.

No capability needs knowledge of the others.

Composition Produces Better Boundaries

Inheritance often produces deep hierarchies.

```
Media

↓

Video

↓

Movie

↓

AnimeMovie
```

Responsibility becomes fragmented across multiple types.

Instead:

```
Movie

↓

Metadata

↓

Artwork

↓

Playback

↓

Progress
```

Each concern remains isolated.

Business concepts remain obvious.

Embedding

Struct embedding is a language convenience.

It promotes methods from one type onto another.

Example:

```
[go]
type Job struct {
    *log.Logger
}
```

The `Job` now exposes the logger's methods directly.

However:

The logger still exists as a field.

It has **not** become a superclass.

Embedding should therefore be viewed as syntactic convenience rather than inheritance.
[oai_citation:1‡Go](https://go.dev/doc/effective_go?utm_source=chatgpt.com)

When To Embed

Embedding SHOULD only be used when:

- the embedded type is conceptually part of the containing type
- method promotion improves readability
- ownership remains obvious

Examples include:

- loggers
- mutexes
- reusable infrastructure helpers

Embedding should improve ergonomics.

Not architecture.

When Not To Embed

Avoid embedding purely to imitate inheritance.

Poor:

```
[go]
type UserService struct {
    BaseService
}
```

Questions immediately arise.

What is a BaseService?

Why does every service inherit it?

What behaviour does it actually own?

This pattern usually indicates an object-oriented design has been translated directly into Go.

Instead:

```
[go]
type UserService struct {
    logger *slog.Logger
    events Publisher
}
```

Dependencies become explicit.

Responsibilities remain clear.

Polymorphism

Polymorphism allows different implementations to satisfy the same behaviour.

Go achieves this through interfaces.

Example:

```
[go]
type Storage interface {
    Save(context.Context, Media) error
}
```

Multiple implementations may exist.

```
Postgres

↓

Storage

DuckDB

↓

Storage

Memory

↓

Storage
```

The service does not care which implementation it receives.

It depends only upon behaviour.

Behaviour Before Type

Within Mosaic, engineers should think:

> *I need something that can publish events.*

Not:

> *I need a Kafka publisher.*

The implementation becomes a deployment decision.

The behaviour remains constant.

Composition Enables Polymorphism

Composition and interfaces work together.

```
Service

↓
```

```
Publisher Interface

↓

NATS Publisher
```

or

```
Service

↓

Publisher Interface

↓

Memory Publisher
```

The service remains unchanged.

Only composition changes.

This is one of Go's greatest strengths.

Avoid Base Types

The following patterns SHOULD NOT exist.

```
BaseService

BaseRepository

AbstractHandler

AbstractProvider
```

These names rarely communicate genuine responsibilities.

Instead, they accumulate unrelated behaviour until they become impossible to reason about.

If multiple types genuinely require shared behaviour, extract a reusable component rather than inventing a base type.

Favour Delegation

Suppose several services require metrics.

Avoid:

```
[text]
BaseService

↓

Metrics

↓

Logging

↓

Tracing
```

Prefer:

```
[go]
type Service struct {
    metrics Metrics
}
```

Delegation keeps ownership explicit.

Every dependency remains visible.

Reuse Behaviour, Not Identity

Inheritance answers:

> *"Is this an X?"*

Composition answers:

> *"Can this perform X?"*

The second question usually produces better software.

Components become reusable because of what they do.

Not because of where they appear in a hierarchy.

Mosaic Guidelines

Within Mosaic:

- Composition MUST be preferred over inheritance.
- Embedding SHOULD only be used where it improves ergonomics.
- Embedding MUST NOT be used to simulate inheritance.
- Shared behaviour SHOULD become reusable components.
- Interfaces provide polymorphism.
- Components collaborate through behaviour rather than hierarchy.
- Base classes and abstract classes are prohibited.

Decision Checklist

Before embedding or introducing shared behaviour, ask:

1. Am I reducing duplication or creating hierarchy? 2. Would explicit composition be easier to understand? 3. Does embedding genuinely improve readability? 4. Does this introduce hidden behaviour? 5. Would another engineer immediately understand the relationship?

If the answer to any question introduces doubt, explicit composition should generally be preferred.

Summary

Composition is the architectural foundation of Go.

Embedding is a convenience.

Interfaces enable polymorphism.

Each solves a different problem.

Keeping these concepts separate allows Mosaic to produce software that is modular, testable and resilient to change without inheriting the complexity traditionally associated with object-oriented inheritance.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`06-interfaces-and-abstraction.md`

Next File

`08-error-handling.md`

Error Handling

> *Errors are not exceptional. They are an expected outcome of software interacting with the real world.*

Purpose

Go treats errors as values.

Unlike languages that separate normal execution from exceptional execution, Go requires engineers to acknowledge failure explicitly.

Within Mosaic, error handling is considered part of the application's public behaviour.

An error should communicate:

- what failed
- why it failed
- whether recovery is possible
- enough context to diagnose the problem

Nothing more.

Nothing less.

Philosophy

Within Mosaic:

> ***Errors are part of the API.***

Returning an error is not admitting failure.

It is communicating reality.

Every function capable of failing should communicate that possibility honestly.

Ignoring failure is considered a defect.

Errors Are Values

Errors should be treated like every other value in Go.

They should be:

- returned
- wrapped
- inspected
- propagated
- transformed

They should not be hidden.

They should not be ignored.

Handle Errors Immediately

Errors SHOULD be handled at the point they are encountered.

Preferred:

```
[go]
user, err := repo.Find(ctx, id)
if err != nil {
    return nil, fmt.Errorf("find user: %w", err)
}
```

Avoid:

```
[go]
user, err := repo.Find(ctx, id)

// 50 lines later...

if err != nil {
    ...
}
```

The further an error travels before being acknowledged, the harder it becomes to understand.

Never Ignore Errors

The following is prohibited.

```
[go]
value, _ := parser.Parse(data)
```

Likewise:

```
[go]
_ = file.Close()
```

Every ignored error must be an intentional engineering decision.

If an error is intentionally ignored, a comment SHOULD explain why.

Example:

```
[go]
// Best effort cleanup.
_ = os.Remove(tempFile)
```

Add Context

Errors should accumulate context as they move upwards.

Poor:

```
[go]
return err
```

Better:

```
[go]
return fmt.Errorf("load metadata: %w", err)
```

Even better:

```
[go]
return fmt.Errorf("load metadata for %q: %w", mediaID, err)
```

Context should describe the operation that failed.

Not merely repeat the underlying error.

Wrap Errors Deliberately

Wrapping an error exposes it to callers through `errors.Is` and `errors.As`.

Example:

```
[go]
return fmt.Errorf("save media: %w", err)
```

Callers may now inspect the underlying error.

This is a deliberate API decision.

Do **not** wrap every error automatically.

If exposing the underlying implementation would leak internal details or constrain future changes, prefer adding context without wrapping. The Go team recommends using `%w` only when you intentionally want callers to inspect the underlying error.
[oai_citation:0‡Go](https://go.dev/blog/go1.13-errors?utm_source=chatgpt.com)

Check Errors Using errors.Is

Sentinel errors MUST be checked using:

```
[go]
errors.Is(err, ErrNotFound)
```

Never:

```
[go]
err == ErrNotFound
```

Wrapped errors remain discoverable through `errors.Is`.

This keeps error handling resilient as additional context is added.

Check Error Types Using errors.As

Custom error types SHOULD be inspected using:

```
[go]
var validation *ValidationError

if errors.As(err, &validation) {
    ...
}
```

Never:

```
[go]
validation := err.(*ValidationError)
```

or

```
[go]
switch err.(type)
```

when wrapped errors are possible.

Sentinel Errors

Sentinel errors represent known business conditions.

Example:

```
[go]
var ErrNotFound = errors.New("not found")
```

Good uses include:

- not found
- already exists
- permission denied
- invalid credentials

Sentinel errors SHOULD represent stable concepts.

They SHOULD NOT represent implementation details.

Custom Error Types

Custom error types SHOULD only exist when additional information is genuinely required.

Example:

```
[go]
type ValidationError struct {
    Field string
    Reason string
}
```

Additional structured data is appropriate.

Merely changing the error message is not.

Error Messages

Error messages SHOULD:

- begin with lowercase
- avoid punctuation
- describe what happened
- avoid repeating caller context

Good:

```
connection refused
```

```
media not found
```

Poor:

```
Error:
```

```
An unexpected error occurred.
```

```
MetadataService failed because...
```

The caller already knows which service produced the error.

Error messages should compose naturally when wrapped.

Log Errors Once

Errors SHOULD be logged exactly once.

Typically this occurs at the application's boundary.

Examples include:

- HTTP middleware
- Worker entry point
- CLI command
- Scheduler
- Background processor

Lower layers SHOULD return errors.

They SHOULD NOT log them.

Otherwise identical failures become logged multiple times.

Panic

`panic` is **not** an error handling mechanism.

Within Mosaic, panic is reserved for situations where the application cannot safely continue.

Examples include:

- impossible internal states
- programmer errors
- unrecoverable startup failures

Panics SHOULD NOT be used for:

- validation
- network failures
- database errors

- user input
- business logic

Most applications should execute for months without producing a panic.

Recover

`recover` SHOULD only exist at application boundaries.

Examples:

- HTTP server middleware
- Worker runtime
- Scheduler

Recovering from panics deep within business logic hides defects.

Instead:

Crash the current operation.

Log sufficient diagnostic information.

Continue serving other requests where appropriate.

Translate Errors

Different architectural layers speak different languages.

Example:

```
sql.ErrNoRows

↓

Repository

↓

ErrMediaNotFound

↓

Service

↓

HTTP 404
```

Business logic should never depend upon SQL errors.

Transport should never depend upon database drivers.

Each layer translates errors into concepts meaningful to the layer above.

Error Boundaries

Every layer has clear responsibilities.

Layer	Responsibility
Infrastructure	Return implementation errors
Repository	Translate infrastructure into domain errors
Service	Add business context
Transport	Convert errors into HTTP, CLI or API responses
Middleware	Log and observe failures

Each layer adds value.

None should duplicate responsibility.

Avoid String Comparisons

The following is prohibited.

```
[go]
if err.Error() == "record not found" {
    ...
}
```

Error messages are for humans.

Program logic should rely upon:

- `errors.Is`
- `errors.As`
- well-defined sentinel errors
- custom error types

Never textual comparison.

Anti-Patterns

The following practices are prohibited.

Ignoring Errors

```
[go]
_, _ = writer.Write(data)
```

Logging Then Returning

```
[go]
log.Error(err)

return err
```

Panic For Business Logic

```
[go]
panic("user not found")
```

Comparing Error Strings

```
[go]
if err.Error() == ...
```

Wrapping Everything

```
[go]
fmt.Errorf("db: %w", sql.ErrNoRows)
```

when the caller should never know a SQL database exists.

Every wrapped error becomes part of your API surface.

Mosaic Guidelines

Within Mosaic:

- Every returned error MUST be handled.
- Errors SHOULD gain context as they propagate.
- Errors MUST NOT be ignored silently.
- Lower layers MUST NOT log errors.
- Errors SHOULD be logged once.
- Business logic SHOULD define business errors.
- `errors.Is` MUST replace equality checks.
- `errors.As` MUST replace type assertions where wrapping is possible.
- Panic is reserved for unrecoverable programmer failures.

Summary

Error handling is communication.

Every error should answer:

- What failed?
- Why did it fail?
- Can the caller recover?
- Is there enough context to diagnose the problem?

If those questions are answered clearly, the error has fulfilled its purpose.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`07-composition-and-polymorphism.md`

Next File

`09-context-and-cancellation.md`

Context and Cancellation

> *A context represents the lifetime of work. It is not a bag of dependencies, a configuration object or a convenient place to hide state.*

Purpose

`context.Context` is one of the most important types in Go.

It allows unrelated parts of an application to agree on one thing:

> ***When should this work stop?***

Within Mosaic, every request, background task and long-running operation must have a clearly defined lifetime.

Context provides the mechanism by which that lifetime is communicated.

Philosophy

Within Mosaic:

> ***Context owns cancellation. It owns nothing else.***

Context exists to communicate:

- cancellation
- deadlines
- timeouts
- request-scoped values

It does **not** exist to transport dependencies through an application.

Why Context Exists

Modern applications perform many concurrent operations.

Consider an HTTP request.

```
HTTP Request

↓

Authentication

↓

Database Query

↓

Metadata Lookup

↓
```

```
Blob Storage
↓
Event Publishing
```

If the client disconnects halfway through, every remaining operation should stop.

Without context:

- goroutines continue running
- database queries continue executing
- HTTP calls continue waiting
- resources remain allocated

With context:

One cancellation propagates throughout the entire operation.

This is precisely the design intent of Go's `context` package.
[oai_citation:0‡Go](https://go.dev/blog/context?utm_source=chatgpt.com)

Context Lifecycle

Every request begins with a root context.

```
Incoming Request
↓
Context Created
↓
Passed Down
↓
Derived Where Necessary
↓
Cancelled
↓
All Child Operations Exit
```

Contexts form a tree.

Cancelling a parent automatically cancels every child.

Child contexts must never outlive their parent unless this behaviour is explicitly required.

Context Is Always The First Parameter

Every function requiring a context MUST accept it as its first parameter.

Example:

```
[go]
func (s *Service) FindMedia(
    ctx context.Context,
    id string,
) (*Media, error)
```

Never:

```
[go]
func (s *Service) FindMedia(
    id string,
    ctx context.Context,
)
```

The ordering should remain consistent throughout the entire codebase.

This is an established Go convention. [oai_citation:1‡Go Packages](https://pkg.go.dev/context?utm_source=chatgpt.com)

Never Store Context In Structs

The following is prohibited.

```
[go]
type Service struct {
    ctx context.Context
}
```

Contexts are request-scoped.

Services are application-scoped.

Mixing the two creates subtle lifecycle bugs.

Instead:

```
[go]
func (s *Service) Process(
    ctx context.Context,
)
```

Every operation receives its own context.

The service remains stateless.

The Go package documentation explicitly advises against storing contexts in structs. [oai_citation:2‡Go Packages](https://pkg.go.dev/context?utm_source=chatgpt.com)

Never Pass nil

A context must always be supplied.

Poor:

```
[go]
Process(nil)
```

Preferred:

```
[go]
Process(context.Background())
```

or

```
[go]
Process(context.TODO())
```

Nil contexts create unnecessary defensive programming.

Create Context At Boundaries

Contexts should be created only at application boundaries.

Examples include:

- HTTP handlers
- CLI commands
- Workers
- Schedulers
- Application startup

Business logic should receive contexts.

It should rarely create them.

Example:

```
HTTP Handler

↓

Service

↓

Repository

↓

Database
```

The handler creates the context.

Everything else propagates it.

Always Propagate Context

Every function receiving a context SHOULD pass it to downstream operations.

Example:

```
[go]
media, err := repo.Find(ctx, id)
```

Not:

```
[go]
media, err := repo.Find(context.Background(), id)
```

Replacing a caller's context breaks cancellation propagation.

Never discard an existing context without a compelling architectural reason.

Derive Context Sparingly

Child contexts should only be created when introducing:

- a timeout
- a deadline
- manual cancellation
- request-scoped values

Example:

```
[go]
ctx, cancel := context.WithTimeout(
    ctx,
    5*time.Second,
)
defer cancel()
```

Creating unnecessary context hierarchies makes cancellation harder to understand.

Always Call Cancel

Whenever `WithCancel`, `WithTimeout` or `WithDeadline` is used, the returned cancel function MUST be called.

Example:

```
[go]
ctx, cancel := context.WithTimeout(
    ctx,
    time.Second,
)
defer cancel()
```

Calling `cancel` releases resources associated with the derived context, even if the timeout never expires. Failing to do so may leak timers and child contexts. [oai_citation:3‡Go Packages](https://pkg.go.dev/context?utm_source=chatgpt.com)

Respect Cancellation

Long-running operations SHOULD regularly check whether cancellation has been requested.

Example:

```
[go]
select {

case <-ctx.Done():
    return ctx.Err()

default:
}
```

Cancellation should terminate work as quickly as practical.

Ignoring cancellation wastes resources.

Context Values

`context.WithValue` exists for request-scoped metadata.

Examples include:

- request IDs
- correlation IDs
- trace IDs
- authenticated principals

It SHOULD NOT be used for:

- configuration
- services
- repositories
- loggers
- feature flags
- optional parameters

Context values communicate request metadata.

Not dependencies.

Background Context

`context.Background()` represents the root of a context tree.

It SHOULD only appear in:

- `main()`
- tests
- application startup
- root workers

Business logic should almost never construct a background context.

Receiving a context is almost always preferable.

TODO Context

`context.TODO()` indicates:

> *"A context should exist here, but the correct one has not yet been determined."*

It is acceptable during development.

It SHOULD NOT remain in production code indefinitely.

Timeouts

Timeouts belong at system boundaries.

Examples include:

- HTTP clients
- database queries
- external APIs
- object storage
- message brokers

Business logic generally should not impose arbitrary timeouts.

The caller owns the operation's lifetime.

Background Work

One of the most common mistakes is using a request context for background processing.

Poor:

```
HTTP Request

↓

Spawn Goroutine

↓

Request Completes

↓

Context Cancelled

↓

Background Work Stops
```

If work must continue after the request ends, a new context with an appropriate lifetime should be created deliberately.

This decision should be rare and well documented.

Graceful Shutdown

Application shutdown should propagate through context.

Typical flow:

```
SIGTERM

↓

Root Context Cancelled

↓
```

```
Workers Stop

↓

Requests Finish

↓

Resources Closed

↓

Application Exits
```

Every long-running component should honour context cancellation.

This enables predictable shutdown behaviour across the entire platform.
[oai_citation:4‡Go](https://go.dev/doc/database/cancel-operations?utm_source=chatgpt.com)

Anti-Patterns

The following practices are prohibited.

Storing Context In Structs

```
[go]
type Repository struct {
    ctx context.Context
}
```

Passing nil

```
[go]
Process(nil)
```

Creating Background Context Deep In Business Logic

```
[go]
context.Background()
```

inside services or repositories.

Using Context As Dependency Injection

```
[go]
ctx.Value("database")
```

Ignoring ctx.Done()

Long-running operations that never respond to cancellation.

Forgetting cancel()

```
[go]
ctx, cancel := context.WithTimeout(...)

// Missing:

cancel()
```

Mosaic Guidelines

Within Mosaic:

- Context MUST be the first parameter.
- Context MUST NOT be stored in structs.
- Context MUST always be propagated.
- Business logic SHOULD NOT create root contexts.
- `cancel()` MUST always be called.
- Context values MUST only contain request-scoped metadata.
- Long-running operations MUST respect cancellation.
- Background work MUST define its own lifetime explicitly.

Summary

Context is not merely another function parameter.

It defines the lifetime of work.

Every operation within Mosaic should be able to answer:

- Who owns this context?
- When will it be cancelled?
- What happens when cancellation occurs?
- Are downstream operations respecting it?

If those questions have clear answers, the application will naturally become more responsive, more efficient and easier to shut down safely.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`08-error-handling.md`

Next File

`10-concurrency.md`

Concurrency

> *Concurrency is about structuring software to perform multiple tasks independently. Parallelism is merely one possible outcome.*

Purpose

Concurrency is one of Go's defining strengths.

It allows software to perform multiple independent operations without blocking overall progress.

Within Mosaic, concurrency underpins almost every major subsystem.

Examples include:

- Metadata ingestion
- Event processing
- Background indexing
- Cache refresh
- Playback monitoring
- Scheduler execution
- Blob synchronisation
- Docker orchestration
- Extension communication

For this reason, concurrency is considered an architectural concern rather than merely a language feature.

Philosophy

Within Mosaic:

> ***Concurrency exists to improve responsiveness, not complexity.***

Every goroutine introduces:

- another execution path
- another lifecycle
- another failure mode
- another debugging challenge

Concurrency should therefore be introduced deliberately.

Not automatically.

Concurrency vs Parallelism

These terms are frequently confused.

They describe different concepts.

Concurrency	Parallelism
Structuring independent work	Executing work simultaneously
Improves responsiveness	Improves throughput
Language concern	Hardware concern

A concurrent application is not necessarily parallel.

A parallel application is usually concurrent.

Go provides excellent support for both.

When To Use Concurrency

Concurrency SHOULD be introduced when:

- waiting on I/O
- independent work can proceed simultaneously
- latency can be reduced
- responsiveness improves
- background processing is appropriate

Examples include:

```
Metadata Lookup

+

Artwork Download

+

Subtitle Discovery
```

These operations do not depend upon one another.

Running them concurrently improves overall response time.

When Not To Use Concurrency

Concurrency SHOULD NOT be introduced when:

- operations are dependent
- work is CPU trivial
- sequential execution is easier to understand
- added complexity outweighs performance gains

Poor:

```
Validate
↓
Spawn Goroutine
↓
Wait
↓
Continue
```

Nothing has been gained.

Sequential execution is clearer.

Goroutine Ownership

Every goroutine MUST have a clearly defined owner.

Ownership answers:

- Who started it?
- Who stops it?
- Who waits for it?
- Who handles failures?
- Who cleans up resources?

If these questions cannot be answered, the goroutine should not exist.

Goroutine Lifecycle

Every goroutine should follow the same lifecycle.

```
Created
↓
Running
↓
Cancellation Requested
↓
Cleanup
↓
Exit
```

No goroutine should run indefinitely without an explicit architectural reason.

Never Fire And Forget

The following is prohibited.

```
[go]
go process()
```

without:

- lifecycle management
- cancellation
- error handling
- ownership

Every goroutine should be observable.

If work is important enough to execute, it is important enough to manage.

Use errgroup For Related Work

When multiple goroutines contribute towards a single operation, `errgroup` SHOULD be preferred.

Example:

```
Metadata

+

Artwork

+

Subtitles

↓

Combined Result
```

Benefits include:

- automatic cancellation
- error propagation
- coordinated completion

This is simpler than manually coordinating channels and wait groups.

The `errgroup` package is widely recommended for managing groups of related goroutines. ([pkg.go.dev](https://pkg.go.dev/golang.org/x/sync/errgroup?utm_source=chatgpt.com))

Worker Pools

Repeated background work SHOULD use worker pools.

Example:

```
Queue
```

```
↓

Worker 1

Worker 2

Worker 3

↓

Completed Tasks
```

Worker pools provide:

- predictable resource usage
- backpressure
- easier monitoring
- bounded concurrency

Creating thousands of goroutines for identical work should generally be avoided.

Channels

Channels communicate ownership.

They are not simply queues.

Before introducing a channel ask:

> *Does another goroutine genuinely need to communicate?*

If not, a normal function call is probably sufficient.

Channels should communicate:

- work
- completion
- cancellation
- events

They should not replace normal data structures.

Channel Direction

Whenever practical, channel direction SHOULD be specified.

Example:

```
[go]
func Publish(events chan<- Event)

[go]
func Consume(events <-chan Event)
```

Directional channels communicate intent.

The compiler prevents accidental misuse.

Buffered Channels

Buffered channels should have a clearly documented purpose.

Good reasons include:

- absorbing bursts
- smoothing producer / consumer rates
- bounded queues

Poor reason:

```
It fixed a deadlock.
```

Buffer size is an architectural decision.

Not a debugging tool.

Closing Channels

Only the sender closes a channel.

Receivers MUST NOT close channels they do not own.

Closing communicates:

> *No more values will ever be sent.*

It is not a signal that processing has completed.

Select Statements

Long-running goroutines SHOULD use `select`.

Example:

```
[go]
select {

case <-ctx.Done():

case event := <-events:
}
```

This allows:

- cancellation
- responsiveness
- graceful shutdown

Ignoring `ctx.Done()` in long-running loops is prohibited.

Shared Memory

Go encourages communicating through channels.

However:

Mutexes are entirely appropriate when protecting shared state.

Do not force channel-based designs where simple locking is clearer.

The Go proverb "Do not communicate by sharing memory; instead, share memory by communicating" is guidance rather than an absolute rule. Simpler synchronisation mechanisms should be chosen where they better fit the problem.
([go.dev](https://go.dev/blog/share-memory-by-communicating?utm_source=chatgpt.com))

Mutexes

A mutex protects ownership of mutable state.

Example:

```
[text]
Cache

↓

Mutex

↓

Read

Write
```

Mutexes SHOULD remain private.

Callers should never need to understand locking behaviour.

RWMutex

`sync.RWMutex` SHOULD only be introduced when measurements demonstrate significant read contention.

It is not automatically faster than `sync.Mutex`.

Additional complexity requires measurable benefit.

Atomics

Atomic operations SHOULD be reserved for:

- counters
- flags
- metrics
- simple state transitions

If multiple values must remain consistent together, prefer a mutex.

Backpressure

Every concurrent pipeline should have a strategy for overload.

Examples include:

- bounded queues
- worker pools
- rate limiting
- request rejection

Unlimited buffering is prohibited.

Every queue should have a maximum capacity.

Long Running Services

Long-running services should follow a common lifecycle.

```
Start

↓

Wait

↓

Process

↓

Cancellation

↓

Cleanup

↓

Exit
```

Every service should honour context cancellation.

Shutdown should be graceful.

Concurrency Patterns

Within Mosaic the following patterns are encouraged.

- Worker Pool
- Fan-Out / Fan-In
- Pipeline

- Event Loop
- Producer / Consumer
- Stale-While-Revalidate
- Background Scheduler

Future specifications explore each pattern individually.

Anti-Patterns

The following practices are prohibited.

Fire And Forget Goroutines

```
[go]
go process()
```

with no ownership.

Unbounded Goroutines

```
for {

go process()
}
```

Ignoring Context

Long-running goroutines that never observe cancellation.

Closing Someone Else's Channel

Receivers closing channels they did not create.

Channels As Databases

Using channels to permanently store application state.

Shared Mutable State Without Synchronisation

Concurrent access to mutable memory without:

- mutexes
- atomics
- channels

The race detector should never report data races in production code. Running tests with `-race` is an important part of validating concurrent programs.
([go.dev](https://go.dev/doc/articles/race_detector?utm_source=chatgpt.com))

Mosaic Guidelines

Within Mosaic:

- Every goroutine MUST have an owner.
- Every goroutine MUST honour cancellation.
- Fire-and-forget goroutines are prohibited.
- Worker pools SHOULD be used for repeated background work.
- `errgroup` SHOULD coordinate related concurrent tasks.
- Shared mutable state MUST be synchronised.
- Queues MUST be bounded.
- Shutdown MUST be graceful.

Concurrency should reduce waiting.

It should never reduce understanding.

Summary

Concurrency is one of Go's greatest strengths.

It is also one of its easiest features to misuse.

Within Mosaic, concurrency is considered successful when:

- ownership is obvious
- cancellation is respected
- resources are bounded
- shutdown is graceful
- debugging remains straightforward

A concurrent system should feel predictable.

Not mysterious.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`09-context-and-cancellation.md`

Next File

`11-testing.md`

Testing

> *Testing is not about proving software works. It is about proving software continues to work after change.*

Purpose

Software that cannot be tested cannot be changed with confidence.

Within Mosaic, testing is considered an architectural responsibility rather than a development afterthought.

Well-designed software naturally becomes easy to test.

Poorly-designed software usually requires increasingly complex tests to compensate for architectural deficiencies.

This document defines the testing philosophy and standards for all Go code within the Mosaic ecosystem.

Philosophy

Within Mosaic:

> ***Design for testability, not test around poor design.***

When code becomes difficult to test, the first question should be:

> *"Is the design wrong?"*

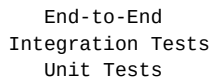
Not:

> *"How do we mock this?"*

Difficult tests often indicate architectural problems rather than testing problems.

The Testing Pyramid

Mosaic adopts the classic testing pyramid.



The majority of tests should exist at the unit level.

Integration tests verify architectural boundaries.

End-to-end tests verify complete workflows.

Each level answers different questions.

Test Responsibilities

Test Type	Purpose
Unit Test	Verify behaviour of one component
Integration Test	Verify collaboration between components
End-to-End Test	Verify complete user workflows
Benchmark	Measure performance
Race Test	Detect concurrency defects

No single type of test replaces another.

Unit Tests

Unit tests SHOULD:

- execute quickly
- remain deterministic
- avoid external services
- avoid network access
- avoid databases
- avoid shared state

A unit test should exercise one unit of behaviour.

Not an entire application.

Integration Tests

Integration tests verify collaboration between components.

Examples include:

```
Repository

↓

PostgreSQL

Metadata

↓

DuckDB

HTTP

↓

Authentication

↓

Service
```

```
↓
Repository
```

Real infrastructure SHOULD be used whenever practical.

Mocking databases during integration testing defeats the purpose.

End-to-End Tests

End-to-end tests exercise complete workflows.

Example:

```
HTTP Request
↓
Authentication
↓
Metadata
↓
Playback
↓
Database
↓
Response
```

These tests provide the highest confidence.

They also provide the highest maintenance cost.

They should therefore remain focused on critical user journeys.

Test Behaviour

Tests MUST verify behaviour.

They SHOULD NOT verify implementation details.

Poor:

```
Did function A call function B?
```

Better:

```
Was the expected behaviour observed?
```

Implementation changes should rarely require test changes.

Behaviour changes should.

Test Naming

Tests SHOULD describe behaviour.

Good:

```
[go]
func TestLibraryReturnsMediaByID(t *testing.T)
```

Better:

```
[go]
func TestFindReturnsNotFoundWhenMediaDoesNotExist(t *testing.T)
```

The test name should describe the expected outcome.

Table-Driven Tests

Whenever multiple scenarios exercise identical behaviour, table-driven tests SHOULD be used.

Example:

```
[go]
tests := []struct {
    name string
    input string
    expected error
}{
    ...
}
```

Benefits include:

- reduced duplication
- improved readability
- easier extension
- consistent structure

Table-driven testing is an established Go convention.
([go.dev](https://go.dev/wiki/TableDrivenTests?utm_source=chatgpt.com))

Test Independence

Tests MUST NOT depend upon:

- execution order
- global state
- previous tests
- shared files
- shared databases

Every test should be executable independently.

Parallel execution should not change results.

Avoid Over-Mocking

Mocks have value.

Over-mocking does not.

Poor:

```
HTTP

↓

Mock Service

↓

Mock Repository

↓

Mock Database

↓

Mock Cache
```

Nothing meaningful is being exercised.

Instead:

Mock only the dependency whose behaviour is irrelevant to the current test.

Everything else should remain real.

Fakes Over Mocks

Where practical, prefer lightweight fake implementations.

Example:

```
[go]
type FakeRepository struct {
    media map[string]Media
}
```

Fakes are often:

- easier to understand
- deterministic
- reusable
- maintenance friendly

Mock frameworks should not become architectural dependencies.

Assertions

Assertions SHOULD verify observable behaviour.

Good:

```
[go]
if got != expected {
    t.Fatalf(...)
}
```

Avoid unnecessary assertion libraries unless they clearly improve readability.

The standard library should remain sufficient for most tests.

Golden Files

Golden files SHOULD be used when verifying:

- generated JSON
- generated Markdown
- templates
- rendered output
- API specifications

Golden files reduce duplication while making expected output easy to review.

Updates should always be intentional.

Benchmarking

Performance assumptions MUST be validated through benchmarks.

Example:

```
[go]
func BenchmarkParser(b *testing.B)
```

Benchmarks should measure:

- allocations
- throughput
- latency

Optimisation without measurement is prohibited.

Race Detection

Every repository SHOULD regularly execute:

```
go test -race ./...
```

The race detector identifies concurrent access to shared memory before defects reach production.

Concurrent code without race testing should be considered incomplete. The Go race detector is specifically designed to uncover data races during testing. ([go.dev](https://go.dev/doc/articles/race_detector?utm_source=chatgpt.com))

Coverage

Code coverage is an indicator.

It is not a goal.

100% coverage can still produce poor software.

Focus on:

- meaningful behaviour
- critical paths
- failure cases
- boundary conditions

Not percentages.

Coverage should support engineering judgement.

It should never replace it.

What To Test

Prioritise testing:

- business rules
- edge cases
- failure scenarios
- concurrency
- validation
- event ordering
- state transitions

These areas provide the highest engineering value.

What Not To Test

Avoid testing:

- trivial getters
- language features
- third-party libraries
- generated code
- implementation details

Trust the language.

Test your behaviour.

Continuous Integration

Every pull request SHOULD execute:

```
Formatting

↓

Static Analysis

↓

Unit Tests

↓

Integration Tests

↓

Race Detector

↓

Benchmarks (optional)

↓

Build
```

No code should be merged without automated verification.

Test Data

Test data SHOULD remain:

- local
- deterministic
- representative
- minimal

Large fixtures should only exist when they significantly improve readability.

Anti-Patterns

The following practices are prohibited.

Sleep-Based Tests

```
[go]
time.Sleep(...)
```

Waiting should be synchronised explicitly.

Testing Private Functions

Private functions should generally be exercised through exported behaviour.

Testing implementation details creates brittle tests.

Shared Mutable Fixtures

Tests modifying shared state introduce hidden coupling.

Mocking Everything

Over-mocking verifies architecture diagrams rather than software behaviour.

Ignoring Failure Paths

Every meaningful failure path deserves at least one test.

Happy-path-only testing creates false confidence.

Mosaic Guidelines

Within Mosaic:

- Behaviour MUST be tested.
- Unit tests SHOULD remain fast.
- Integration tests SHOULD use real infrastructure where practical.
- Table-driven tests SHOULD be preferred for repeated scenarios.
- Fakes SHOULD be preferred over complex mock frameworks.
- Race detection SHOULD be part of CI.
- Benchmarks SHOULD accompany performance-sensitive code.
- Coverage SHOULD inform discussion rather than drive development.

Summary

Testing is a design activity.

Good tests emerge naturally from good architecture.

When testing becomes painful, engineers should first question the architecture rather than the testing framework.

Within Mosaic, software is considered complete only when its behaviour can be verified confidently and repeatedly through automated tests.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`10-concurrency.md`

Next File

`12-performance.md`

Performance

> *Performance is an architectural characteristic, not a coding style. Optimise systems by understanding them, not by guessing.*

Purpose

Performance is an important quality attribute of every Mosaic service.

However, performance is not the primary objective.

Correctness always comes first.

Maintainability comes second.

Only once software is correct and maintainable should optimisation become a priority.

This document defines how performance should be approached throughout the Mosaic ecosystem.

Philosophy

Within Mosaic:

> ***Measure first. Optimise second.***

Performance work without measurements is engineering theatre.

Optimisations should be driven by:

- benchmarks
- profiling
- production telemetry
- evidence

Never by intuition alone.

Performance Hierarchy

Performance decisions should follow the same order.

```
Correctness
↓
Readability
↓
Maintainability
↓
Measurement
↓
```

```
Optimisation
```

Optimising incorrect software simply produces incorrect software more quickly.

Premature Optimisation

Premature optimisation is prohibited.

Engineers SHOULD NOT:

- introduce complexity for hypothetical gains
- sacrifice readability
- duplicate logic for tiny performance improvements
- rewrite stable code without evidence

Simple software is often surprisingly fast.

Optimisation should solve demonstrated bottlenecks.

Not imagined ones.

Measure Everything

Before optimising, answer:

- What is slow?
- How slow is it?
- Why is it slow?
- What evidence exists?
- What is the expected improvement?

If these questions cannot be answered, optimisation should not begin.

Profiling

Profiling SHOULD always precede optimisation.

The preferred workflow is:

```
Observe
↓
Profile
↓
Identify Bottleneck
↓
Optimise
```

↓

```
Benchmark
```

↓

```
Repeat
```

Go provides excellent profiling tools.

Examples include:

- CPU profiling
- Heap profiling
- Goroutine profiling
- Mutex profiling
- Block profiling

Profile first.

Guess later.

Benchmark Before And After

Every meaningful optimisation SHOULD include benchmarks.

Example:

```
[text]
Before
```

↓

```
Optimisation
```

↓

```
Benchmark
```

↓

```
Compare
```

An optimisation without measurable improvement should be reconsidered.

Benchmarks should demonstrate value.

Not merely existence.

Allocations

Memory allocations are inexpensive.

Large numbers of unnecessary allocations are not.

Reducing allocations can significantly reduce garbage collection pressure.

However:

Readability remains more important than avoiding a single allocation.

Only optimise allocation-heavy paths identified through profiling.

Escape Analysis

Go automatically determines whether values remain on the stack or escape to the heap.

Engineers SHOULD understand escape analysis.

They SHOULD NOT write unreadable code attempting to outsmart the compiler.

Escape analysis should be inspected using:

```
go build -gcflags="-m"
```

Only optimise escape behaviour when measurements demonstrate heap allocations are a bottleneck. [oai_citation:0‡Golang Design](https://golang.design/under-the-hood/en/part5toolchain/ch15compile/escape/?utm_source=chatgpt.com)

Prefer Values

Small immutable values SHOULD generally be passed by value.

Example:

```
[go]
type Position struct {
    X int
    Y int
}
```

Returning values often allows the compiler to keep them on the stack.

Pointers should be introduced because ownership or mutation requires them.

Not because "objects should always be pointers."

Slices

Slices are lightweight descriptors.

Engineers SHOULD:

- preallocate capacity when known
- reuse slices where practical
- avoid unnecessary copying

Example:

```
[go]
items := make([]Item, 0, expected)
```

Preallocation should only be used when the approximate size is genuinely known.

Avoid arbitrary capacity guesses.

Maps

Maps SHOULD be preallocated when expected size is known.

Example:

```
[go]
cache := make(map[string]Media, expected)
```

This reduces internal resizing.

Again:

Estimate based on real knowledge.

Not speculation.

Strings

Avoid unnecessary string construction.

Poor:

```
[go]
result := a + b + c + d + e
```

Repeated concatenation within loops should generally use:

- `strings.Builder`
- `bytes.Buffer`

Appropriate buffering reduces temporary allocations.

Buffers

Temporary buffers SHOULD be reused when profiling demonstrates significant allocation pressure.

Examples include:

- JSON encoding
- Media parsing
- Image processing
- Metadata ingestion

Simple code should always come first.

Pooling should only be introduced where evidence supports it.

sync.Pool

`sync.Pool` is an advanced optimisation tool.

It SHOULD only be used when:

- temporary objects are allocated frequently
- profiling identifies allocation pressure
- reuse measurably reduces GC overhead

It SHOULD NOT become a general-purpose object cache.

Objects stored in a pool may be discarded during garbage collection, so correctness must never depend upon their continued existence. [oai_citation:1‡Go](https://go.dev/wiki/Performance?utm_source=chatgpt.com)

Avoid Reflection

Reflection introduces:

- allocations
- complexity
- reduced compile-time guarantees
- slower execution

Reflection SHOULD be avoided within performance-critical paths.

Code generation or explicit implementations are generally preferable.

Avoid unsafe

The `unsafe` package SHOULD NOT be used.

Exceptions require:

- measurable performance justification
- architectural review
- comprehensive testing
- clear documentation

Breaking Go's safety guarantees is a significant architectural decision.

Not an optimisation technique.

Database Performance

Performance problems should rarely be solved inside Go first.

Instead examine:

- indexes
- query plans
- batching
- pagination

- network latency

A poor SQL query cannot be fixed through clever Go code.

Network Performance

Network performance is usually dominated by latency rather than CPU.

Prefer:

- batching
- caching
- compression
- connection reuse

before attempting micro-optimisations.

Concurrency

Concurrency is not automatically a performance optimisation.

Poor:

```
Sequential
↓
Concurrent
↓
Wait
```

Nothing has improved.

Concurrency should reduce waiting.

Not merely increase goroutine count.

Garbage Collection

The Go garbage collector is highly optimised.

Engineers SHOULD trust it.

Avoid:

- manual memory management patterns
- premature pooling
- complicated allocation avoidance

Reduce allocation pressure only where profiling identifies GC as a bottleneck.

Caching

Caching should improve overall system performance.

Caching should never hide poor architecture.

Before introducing a cache ask:

- What invalidates it?
- Who owns it?
- How is freshness guaranteed?
- Can correctness be affected?

An incorrect cache is worse than no cache.

Observability

Performance cannot be improved if it cannot be observed.

Every production service SHOULD expose:

- latency
- throughput
- allocation rate
- goroutine count
- memory usage
- request duration

Operational metrics should identify regressions before users do.

Performance Reviews

Every significant optimisation SHOULD answer:

- What was measured?
- What changed?
- Why did it improve?
- How much faster is it?
- Was readability affected?

If the improvement cannot be demonstrated objectively, it should be reconsidered.

Anti-Patterns

The following practices are prohibited.

Optimising Without Profiling

```
"I think this is slow."
```

Premature sync.Pool

Using pooling before allocation pressure exists.

Excessive Pointer Usage

Returning pointers simply because "Go passes everything by reference."

Reflection For Convenience

Reflection replacing explicit code without measurable benefit.

Using unsafe

Breaking type safety to save hypothetical nanoseconds.

Benchmarking Unrealistic Workloads

Benchmarks should represent production behaviour.

Synthetic numbers without context are misleading.

Mosaic Guidelines

Within Mosaic:

- Correctness MUST precede optimisation.
- Profiling MUST precede optimisation.
- Benchmarks SHOULD accompany significant performance work.
- Escape analysis SHOULD inform optimisation decisions.
- `sync.Pool` SHOULD only be introduced after profiling.
- Reflection SHOULD be avoided in hot paths.
- `unsafe` requires architectural approval.
- Production services SHOULD expose performance metrics.

Summary

Performance is the result of good architecture.

It is rarely the result of clever code.

Within Mosaic, engineers optimise by:

- measuring first
- understanding bottlenecks
- improving architecture
- validating improvements
- preserving readability

Fast software is valuable.

Understandable fast software is considerably more valuable.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`11-testing.md`

Next File

`13-design-patterns.md`

Design Patterns

> *Design patterns are proven solutions to recurring problems. They are not architectural goals. Good engineers recognise patterns naturally rather than forcing them into every design.*

Purpose

Software engineering frequently encounters similar architectural problems.

Go provides a small but powerful set of language features that allow these problems to be solved without many of the patterns traditionally found in object-oriented languages.

This document defines the preferred design patterns for the Mosaic ecosystem.

It also explains when they should be used and, equally importantly, when they should not.

Philosophy

Within Mosaic:

> ***Choose the simplest pattern that completely solves the problem.***

Patterns exist to reduce complexity.

If introducing a pattern increases complexity without providing measurable value, the pattern should not be used.

Patterns should emerge naturally from architectural requirements.

Not from habit.

Pattern Selection

Before introducing any design pattern, engineers SHOULD ask:

- What problem does this solve?
- Is there a simpler solution?
- Does Go already solve this naturally?
- Will another engineer immediately recognise the pattern?
- Does the pattern improve maintainability?

If these questions cannot be answered clearly, the pattern should probably not be introduced.

Preferred Patterns

The following patterns are considered first-class architectural patterns within Mosaic.

Constructor Pattern

Purpose

Construct fully initialised objects with explicit dependencies.

Example:

```
[go]
repo := metadata.NewRepository(db)

service := metadata.NewService(repo)

handler := http.NewHandler(service)
```

Benefits

- Explicit dependencies
- Easy testing
- Predictable lifecycle
- No hidden initialisation

This is the default construction pattern throughout Mosaic.

Composition Pattern

Purpose

Combine small components into larger behaviour.

Example:

```
[text]
Library Service

↓

Metadata Provider

↓

Artwork Provider

↓

Event Publisher
```

Composition is the primary architectural mechanism within Mosaic.

Inheritance is never required.

Repository Pattern

Purpose

Separate domain logic from persistence.

Example:

```
[text]
Service

↓

Repository

↓

PostgreSQL
```

Repositories own persistence.

Services own business logic.

Repositories MUST NOT contain business rules.

Adapter Pattern

Purpose

Translate one interface into another.

Typical Mosaic examples include:

```
Jellyfin

↓

Mosaic API

Stremio

↓

Metadata Provider

Docker

↓

Extension Runtime
```

Adapters isolate external systems from internal architecture.

Business logic should never depend directly upon third-party APIs.

Strategy Pattern

Purpose

Allow behaviour to vary without modifying callers.

Example:

```
[text]
Artwork Provider

↓

TMDB
```

```
AniList

Local Storage
```

Each implementation satisfies the same behaviour.

The caller remains unchanged.

In Go, strategies naturally emerge through interfaces rather than inheritance.

Functional Options

Purpose

Configure objects with many optional parameters.

Example:

```
[go]
server := NewServer(

    WithLogger(logger),

    WithMetrics(metrics),

    WithTLS(config),
)
```

Functional options SHOULD be used when:

- optional configuration grows
- constructors become difficult to read
- configuration remains immutable after creation

They SHOULD NOT replace required constructor parameters.

Required dependencies belong in constructors.

Middleware Pattern

Purpose

Wrap behaviour without modifying implementations.

Example:

```
HTTP Request

↓

Logging

↓

Authentication

↓

Metrics
```

↓

```
Compression
```

↓

```
Handler
```

Middleware composes behaviour cleanly.

It should remain focused.

Each middleware should own one responsibility.

Worker Pool Pattern

Purpose

Process bounded concurrent work.

Example:

```
Queue
```

↓

```
Workers
```

↓

```
Results
```

Worker pools prevent unbounded goroutine creation.

They provide predictable throughput and resource usage.

This is the preferred concurrency model for repeated background work.

Pipeline Pattern

Purpose

Break processing into sequential stages.

Example:

```
Read
```

↓

```
Parse
```

↓

```
Transform
```

↓

```
Validate
```

↓

```
Persist
```

Each stage owns one responsibility.

Pipelines improve readability and testability.

They also allow individual stages to become concurrent where appropriate.

Event-Driven Pattern

Purpose

Decouple producers from consumers.

Example:

```
Media Imported

↓

Event Bus

↓

Metadata

↓

Artwork

↓

Search Index

↓

Analytics
```

Events communicate facts.

They do not request actions.

This distinction is fundamental to Mosaic's architecture.

Future specifications define the event model in detail.

State Machine Pattern

Purpose

Represent finite business processes explicitly.

Example:

```
Queued

↓

Processing

↓
```

```
Completed
↓
Archived
```

Avoid scattered boolean flags.

State should always be explicit.

Stale-While-Revalidate

Purpose

Improve responsiveness by serving cached data while refreshing asynchronously.

Example:

```
Cache Hit
↓
Immediate Response
↓
Background Refresh
↓
Cache Updated
```

This pattern is heavily used throughout Mosaic for metadata and artwork.

It significantly improves perceived performance while maintaining freshness.

Retry Pattern

Purpose

Recover from transient failures.

Retries SHOULD only be used for operations that are:

- idempotent
- transient
- expected to succeed later

Retries SHOULD always use:

- exponential backoff
- maximum retry count
- cancellation support

Infinite retries are prohibited.

Circuit Breaker

Purpose

Prevent cascading failures.

Typical use cases include:

- external APIs
- remote storage
- metadata providers
- authentication providers

Circuit breakers improve resilience.

They should be implemented at infrastructure boundaries.

Patterns To Use Sparingly

The following patterns have legitimate uses but should remain uncommon.

Decorator

Generally replaced by middleware.

Builder

Usually unnecessary.

Struct literals or constructors are clearer.

Factory

Simple constructors are almost always sufficient.

Large factory hierarchies indicate over-engineering.

Singleton

Application-scoped dependencies should instead be owned by the composition root.

Global singletons are prohibited.

Patterns To Avoid

The following patterns SHOULD NOT appear within Mosaic.

Abstract Factory

Go's explicit constructors make this unnecessary.

Service Locator

Hidden dependencies reduce readability.

God Object

Large components owning unrelated responsibilities.

Base Classes

Go has no inheritance.

Do not simulate it.

Generic Manager

Packages named:

`Manager`

`Processor`

`Coordinator`

without clearly defined responsibilities.

Reflection-Based Frameworks

Reflection should not become the primary mechanism for application architecture.

Explicit code is easier to understand.

Choosing A Pattern

Every pattern should justify itself.

Ask:

- Does this reduce complexity?
- Does this improve readability?
- Does this reduce coupling?
- Is this idiomatic Go?
- Will future contributors recognise it?

If the answer is "no", another approach should be preferred.

Mosaic Pattern Hierarchy

Patterns are preferred in roughly the following order.

```
Functions

↓

Composition

↓

Interfaces

↓

Constructors

↓

Middleware

↓

Events

↓

Worker Pools

↓

Pipelines

↓

Advanced Patterns
```

Always begin with the simplest possible solution.

Introduce more sophisticated patterns only when genuine architectural requirements emerge.

Summary

Design patterns are tools.

Not objectives.

Within Mosaic:

- Composition is preferred.
- Constructors build systems.
- Interfaces define behaviour.
- Events decouple components.
- Middleware composes behaviour.
- Worker pools manage concurrency.
- Pipelines organise processing.
- Simplicity remains the overriding principle.

The best pattern is the one that disappears into the architecture.

It should make the software feel obvious rather than impressive.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`12-performance.md`

Next File

`14-anti-patterns.md`

Anti-Patterns

> *Technical debt rarely begins with obviously bad decisions. It begins with seemingly convenient decisions that slowly become architectural constraints.*

Purpose

Good engineering is not only about recognising good design.

It is equally important to recognise poor design before it becomes deeply embedded within a system.

This document catalogues the architectural anti-patterns that are prohibited within the Mosaic ecosystem.

Many of these patterns appear attractive initially.

Nearly all become increasingly expensive as software evolves.

Philosophy

Within Mosaic:

> ***Every abstraction carries a maintenance cost.***

Architecture should become simpler over time.

Not more complicated.

Whenever an anti-pattern appears, engineers should ask:

> *"What problem created this?"*

Fixing the underlying cause is almost always preferable to accommodating the symptom.

God Objects

Description

A God Object owns too many unrelated responsibilities.

Example:

```
MediaService

↓

Authentication

Metadata

Playback

Caching

Logging

Scheduling
```

```
Analytics
```

```
Notifications
```

Every new feature eventually finds its way into the same component.

The result is:

- high coupling
- poor cohesion
- merge conflicts
- difficult testing
- slow development

God objects violate the Single Responsibility Principle and are widely recognised as a major software design anti-pattern. [oai_citation:0‡Bitloops](https://bitloops.com/resources/software-design/anti-patterns-in-software-design?utm_source=chatgpt.com)

Symptoms

- Thousands of lines of code.
- Hundreds of methods.
- Large constructor.
- Frequent merge conflicts.
- Difficult to explain responsibility.
- Constant modification.

Preferred Solution

Split responsibilities.

Prefer:

```
Metadata
```

```
Playback
```

```
Search
```

```
Analytics
```

```
Events
```

Each package owns one concern.

God Packages

Description

A package that gradually becomes responsible for unrelated concepts.

Examples:

`common`

`utils`

`shared`

`helpers`

These names communicate convenience rather than ownership.

Eventually every engineer adds another unrelated function.

The package becomes impossible to navigate.

Preferred Solution

Move functionality into the package that owns the behaviour.

Ownership should always be obvious.

Premature Abstraction

Description

Creating abstractions before multiple implementations exist.

Example:

`Repository`

↓

`RepositoryImpl`

when only one repository exists.

Or:

`MediaService`

↓

`MediaServiceInterface`

with exactly one implementation.

This creates additional files without reducing coupling.

Preferred Solution

Build concrete implementations first.

Extract interfaces only after they naturally emerge.

Interface Pollution

Description

Creating interfaces simply because "everything should have one."

Example:

```
Every struct
```

↓

```
Matching interface
```

Large numbers of unused interfaces reduce clarity.

They also encourage unnecessary indirection.

Go developers commonly identify premature interface creation as a frequent source of unnecessary complexity. [oai_citation:1‡Reddit](https://www.reddit.com/r/golang/comments/1oc5is8/writing_better_go_lessons_from_10_code_reviews/?utm_source=chatgpt.com)

Preferred Solution

Interfaces belong to consumers.

Not producers.

Service Locator

Description

Dependencies retrieved dynamically.

Example:

```
[go]
service := container.Resolve("metadata")
```

The dependency graph becomes hidden.

Readers can no longer understand component requirements from constructors.

Problems

- hidden dependencies
- runtime failures
- poor discoverability
- difficult testing

Preferred Solution

Constructor injection.

Every dependency should be visible.

Global Mutable State

Description

Application state exposed globally.

Example:

```
[go]
var Database *sql.DB
```

Every package can modify shared state.

Testing becomes increasingly difficult.

Race conditions become easier to introduce.

Google's Go best practices specifically caution against global package state and global service locators because they obscure ownership and complicate reasoning. [oai_citation:2‡Chromium Git Repositories](https://chromium.googlesource.com/external/github.com/google/styleguide/%2B/HEAD/go/best-practices.md?utm_source=chatgpt.com)

Preferred Solution

Pass dependencies explicitly.

Base Types

Description

Attempting to recreate inheritance.

Examples:

`BaseService`

`AbstractRepository`

`BaseHandler`

These components inevitably accumulate unrelated behaviour.

Eventually every implementation inherits unnecessary functionality.

Preferred Solution

Composition.

Shared behaviour should become reusable components.

Not parent types.

Utility Packages

Description

General-purpose dumping grounds.

Examples:

`utils`

`common`

`helpers`

These packages typically exist because ownership has not been identified.

Preferred Solution

Move code to the package that owns the behaviour.

If ownership cannot be identified, the design probably requires reconsideration.

Boolean Parameters

Description

Functions controlled by flags.

Example:

```
[go]
Process(media, true)
```

The meaning of:

`true`

is invisible.

Adding additional flags rapidly becomes unreadable.

Preferred Solution

Split behaviour into separate functions.

Or introduce explicit configuration types.

Magic Strings

Description

Business logic relying upon hard-coded string literals.

Example:

```
[go]
if status == "completed"
```

Problems include:

- spelling mistakes
- inconsistent values
- poor discoverability
- difficult refactoring

Preferred Solution

Define constants.

Example:

```
[go]
const (
    StatusCompleted = "completed"
)
```

Or preferably:

```
[go]
type Status string
```

This communicates intent through the type system.

Magic Numbers

Description

Literal numeric values without meaning.

Example:

```
[go]
cacheTTL := 3600
```

Questions immediately arise.

3600 what?

Seconds?

Milliseconds?

Minutes?

Preferred Solution

Prefer named constants.

Example:

```
[go]
```

```
const DefaultCacheTTL = time.Hour
```

The intent becomes immediately obvious.

Deep Nesting

Description

Excessive indentation.

Example:

```
if
↓
for
↓
switch
↓
if
↓
select
```

Deep nesting reduces readability.

Preferred Solution

Return early.

Handle error cases immediately.

Flatten control flow wherever practical.

Long Functions

Description

Functions performing many unrelated operations.

Symptoms include:

- scrolling required to understand behaviour
- multiple abstraction levels
- repeated comments
- unrelated responsibilities

Preferred Solution

Extract cohesive behaviour into well-named functions.

Each function should communicate one idea.

Large Constructors

Description

Constructors accepting excessive dependencies.

Example:

```
NewService(

12 parameters
)
```

This usually indicates excessive responsibility.

Preferred Solution

Split the service.

Or introduce cohesive supporting components.

Hidden Goroutines

Description

Constructors starting background work automatically.

Example:

```
[go]
NewCache()
```

Internally:

```
Starts goroutines

Starts timers

Starts cleanup workers
```

Construction should never unexpectedly begin application behaviour.

Preferred Solution

Separate:

```
New()

↓

Start()
```

↓

```
Stop()
```

Lifecycle becomes explicit.

Ignored Errors

Description

Silently discarding errors.

Example:

```
[go]
_, _ = writer.Write(data)
```

Ignored errors create undefined behaviour.

Every error deserves an intentional decision.

Reflection As Architecture

Description

Reflection replacing explicit code.

Examples include:

- runtime dependency injection
- automatic registration
- generic service discovery

Reflection increases:

- runtime failures
- hidden behaviour
- debugging difficulty

Preferred Solution

Explicit construction.

Compile-time guarantees.

Circular Dependencies

Description

Package A depends upon Package B.

Package B depends upon Package A.

The Go compiler prohibits this.

Within Mosaic this is considered an architectural warning rather than an inconvenience.

Preferred Solution

Extract shared behaviour.

Or redesign ownership.

Over-Generalisation

Description

Designing software for hypothetical future requirements.

Examples:

- plugin systems before plugins exist
- generic frameworks before use cases exist
- configuration options nobody uses

Preferred Solution

Solve today's problem well.

Allow tomorrow's abstractions to emerge naturally.

Recognising Anti-Patterns

Most anti-patterns share common characteristics.

They usually:

- increase coupling
- reduce readability
- hide ownership
- introduce indirection
- complicate testing
- make debugging harder

If a proposed solution exhibits several of these characteristics, it deserves additional architectural scrutiny.

Mosaic Guidelines

Within Mosaic:

- God Objects are prohibited.
- God Packages are prohibited.
- Premature abstraction is prohibited.
- Service locators are prohibited.
- Global mutable state is prohibited.
- Base classes are prohibited.
- Utility packages should be avoided.
- Hidden goroutines are prohibited.
- Reflection should not define architecture.
- Magic strings and magic numbers should be replaced with meaningful types or constants.

Good architecture removes accidental complexity.

It never institutionalises it.

Summary

Anti-patterns rarely appear overnight.

They emerge gradually through many individually reasonable decisions.

The responsibility of every engineer is therefore not merely to write good code.

It is to recognise when good code begins drifting towards bad architecture.

The earlier an anti-pattern is identified, the cheaper it is to remove.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`13-design-patterns.md`

Next File

`15-code-review-standards.md`

Code Review Standards

> *The purpose of code review is not to find fault with engineers. It is to improve the quality, consistency and longevity of the software.*

Purpose

Every change made to the Mosaic codebase should improve the software.

Code review exists to ensure that changes are:

- technically correct
- architecturally consistent
- maintainable
- understandable
- testable
- aligned with the Mosaic Engineering Guidelines

Reviews are collaborative engineering activities.

They are not approval ceremonies.

Philosophy

Within Mosaic:

> ***Review the code. Never the author.***

Engineering discussions should focus upon:

- design
- maintainability
- correctness
- readability
- architecture

Personal preference should never outweigh established engineering standards.

When disagreement exists, the MEG should be considered the source of truth.

Objectives

Every code review should answer five questions.

1. Is it Correct?

Does the implementation solve the intended problem?

Does it introduce bugs?

Does it handle failures correctly?

2. Is it Understandable?

Would another engineer immediately understand:

- what the code does
- why it exists
- how it behaves

without additional explanation?

3. Is it Maintainable?

Will this implementation remain understandable in:

- six months
- one year
- five years

Can future requirements be accommodated without major rewrites?

4. Is it Consistent?

Does the implementation follow:

- Go conventions
- Mosaic conventions
- repository conventions

Consistency reduces cognitive load.

5. Is it Simpler?

Does this change reduce complexity?

Or merely move it somewhere else?

Every review should seek opportunities to simplify.

Review Mindset

Reviews should begin with the assumption that:

> *The author made the best decision they could with the information available.*

The objective is to improve the software together.

Not to prove someone wrong.

Review Checklist

Every review SHOULD consider the following.

Correctness

- Does the implementation satisfy the requirements?
- Are edge cases handled?
- Are errors propagated correctly?
- Are resources cleaned up?
- Does cancellation work?
- Are concurrent operations safe?

Architecture

- Does the package own this responsibility?
- Are dependencies flowing correctly?
- Is abstraction appropriate?
- Is composition preferred?
- Has coupling increased?
- Does this align with the MEG?

Readability

- Can names be improved?
- Are functions too large?
- Is the control flow obvious?
- Are comments necessary?
- Would another engineer understand this quickly?

Simplicity

Ask:

Can this become smaller?
Can this become clearer?
Can this become more explicit?

Complexity should always justify itself.

Testing

- Are tests present?
- Do tests verify behaviour?
- Are edge cases covered?
- Are failures tested?
- Does concurrency have tests?
- Would race detection pass?

Performance

- Is optimisation justified?
- Has performance been measured?
- Does complexity outweigh the benefit?
- Is allocation behaviour reasonable?

Premature optimisation should be challenged.

Documentation

- Are exported APIs documented?
- Does architecture documentation require updating?
- Are comments explaining *why* rather than *what*?

Documentation should evolve alongside implementation.

What Reviews Should Not Discuss

The following SHOULD NOT dominate review discussions.

- personal formatting preferences
- preferred variable names without objective improvement
- favourite libraries
- editor configuration
- personal coding style

Automated tooling should handle formatting.

Reviews should focus on engineering.

Review Comments

Comments should be:

- respectful
- specific
- actionable
- educational

Good:

> *This dependency appears to create coupling between playback and metadata. Could the interface belong to the consumer instead?*

Poor:

> *This is wrong.*

Explain the reasoning.

Not merely the conclusion.

Google's Go Code Review Comments encourage reviewers to explain the principle behind requested changes so future code improves as well.
([go.dev](https://go.dev/wiki/CodeReviewComments?utm_source=chatgpt.com))

The Boy Scout Review

Every review should ask one additional question.

> ***Did this pull request leave the surrounding code better than it was before?***

Examples include:

- improved naming
- removed duplication
- simplified logic
- additional tests
- better documentation
- dead code removal

Small improvements accumulate over time.

Review Categories

Review feedback should naturally fall into one of four categories.

Category	Description
Defect	Incorrect behaviour that must be fixed

Category	Description
Architecture	Violates engineering standards or design principles
Suggestion	Improvement that is optional
Question	Clarification requested before approval

Not every comment blocks a merge.

Distinguishing between categories improves collaboration.

Blocking Issues

The following SHOULD block approval.

- Incorrect behaviour
- Race conditions
- Resource leaks
- Security issues
- Broken tests
- Architectural violations
- Hidden dependencies
- Missing error handling
- Unsafe concurrent access
- Significant maintainability concerns

Non-Blocking Issues

The following generally SHOULD NOT block approval.

- Minor naming preferences
- Small formatting issues
- Future optimisation ideas
- Stylistic differences already accepted by the repository

These may be addressed separately.

Self Review

Before requesting review, every engineer SHOULD review their own changes.

Suggested checklist:

- Read every changed line.

- Remove temporary code.
- Remove debugging statements.
- Run formatting.
- Run static analysis.
- Execute tests.
- Read the diff as another engineer would.

The best time to find defects is before someone else has to.

Automated Review

Automation should perform repetitive tasks.

Examples include:

- formatting
- linting
- static analysis
- dependency scanning
- vulnerability detection
- test execution
- race detection

Humans should review:

- architecture
- design
- readability
- maintainability
- intent

Do not spend human attention on problems that tooling can solve reliably.

Large Pull Requests

Large pull requests are difficult to review.

Where practical:

- separate refactoring from feature work
- separate formatting from behaviour changes
- separate infrastructure from business logic

Smaller reviews produce higher quality feedback.

Receiving Feedback

Review feedback should be viewed as an opportunity to improve the software.

Avoid defending implementations automatically.

Instead ask:

- Is the reviewer correct?
- Does this improve the design?
- Is there a simpler solution?

Engineering discussions should optimise for the codebase.

Not individual opinions.

Review Culture

A healthy review culture encourages:

- curiosity
- discussion
- learning
- consistency
- shared ownership

It discourages:

- ego
- blame
- gatekeeping
- personal criticism

The quality of a codebase is directly influenced by the quality of its reviews.

Mosaic Guidelines

Within Mosaic:

- Every significant change SHOULD be reviewed.
- Reviews MUST focus on engineering rather than personal preference.
- Automated tooling SHOULD handle formatting.
- Blocking comments MUST explain the underlying concern.
- Engineers SHOULD perform self-review before requesting review.
- Every review SHOULD leave the surrounding code better than before.

- Architecture discussions SHOULD reference the MEG where applicable.

Summary

Code review is one of the highest leverage engineering activities.

It spreads knowledge.

It maintains consistency.

It prevents architectural drift.

Most importantly, it protects the long-term health of the codebase.

Within Mosaic, the objective of review is simple:

> ***Leave both the software and the engineer better than you found them.***

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`14-anti-patterns.md`

Next File

`16-boy-scout-rule.md`

The Boy Scout Rule

> *Leave the code cleaner than you found it.*

Purpose

Software quality is rarely transformed through large rewrites.

Instead, quality improves through thousands of small, deliberate decisions made by engineers every day.

The Boy Scout Rule provides a simple philosophy for maintaining long-lived software systems.

Every change, regardless of size, should improve the codebase in some measurable way.

Within Mosaic, this principle is considered a core engineering responsibility.

Philosophy

Within Mosaic:

> ***Every commit should leave the repository in a better state than it was found.***

Improvement does not require large refactoring.

It requires continuous attention.

Small improvements made consistently prevent the accumulation of technical debt.

Continuous Improvement

Engineering quality is cumulative.

One engineer:

- improves a variable name

Another:

- removes dead code

Another:

- adds missing tests

Another:

- simplifies a complex function

Months later the software has become significantly easier to maintain.

None of these changes were individually transformative.

Together they fundamentally improve the health of the project.

What Counts As Improvement?

Improvement may include:

- clearer naming
- simpler control flow
- reduced duplication
- smaller functions
- better documentation
- improved package boundaries
- stronger typing
- improved test coverage
- additional benchmarks
- clearer logging
- improved observability
- dead code removal
- removing unnecessary abstraction

Even fixing a spelling mistake contributes positively.

Quality compounds.

Small Changes Matter

Large refactoring efforts are risky.

Small improvements are sustainable.

Example.

Poor:

```
"I'm not touching that because this isn't my feature."
```

Preferred:

```
"I'm already here.

I'll improve this function while I'm modifying it."
```

Improvement should naturally accompany development.

Not wait for dedicated cleanup projects.

Refactoring Is Not Rewriting

The Boy Scout Rule does **not** encourage unnecessary rewrites.

If software is:

- stable
- understandable
- correct

Leave it alone.

Refactor only where improvement provides genuine value.

Every modification carries risk.

Improvement should reduce that risk.

Not increase it.

Opportunistic Refactoring

Refactor code when:

- already modifying the surrounding area
- behaviour is well understood
- tests provide confidence
- architectural improvements are obvious

Avoid unrelated large-scale refactoring during feature development.

Keep changes cohesive.

Incremental Improvement

Examples include:

Before:

```
[go]
func Do(a string, b int) {}
```

After:

```
[go]
func ProcessMedia(
    mediaID string,
    retryCount int,
) {}
```

Behaviour has not changed.

Understanding has improved.

Another example.

Before:

```
[go]
if err != nil {

    return err
}
```

Repeated dozens of times.

After:

Shared behaviour extracted into a focused helper where doing so genuinely improves readability.

Reduce duplication carefully.

Not mechanically.

Naming Is Architecture

One of the highest value improvements an engineer can make is improving names.

Good names reduce documentation.

Good names reduce bugs.

Good names reduce onboarding time.

If a clearer name is discovered while modifying code, it SHOULD be adopted where practical.

Delete Code

Deleting unnecessary code is one of the most valuable forms of improvement.

Dead code:

- increases maintenance cost
- confuses readers
- expands testing surface
- slows refactoring

Unused code SHOULD be removed.

Version control remembers history.

The repository does not need to.

Remove Duplication Carefully

Duplication is not automatically bad.

Sometimes duplication improves clarity.

Extract shared behaviour only when:

- duplication represents the same concept
- abstraction simplifies the design
- future changes become easier

Do not abstract merely because two functions appear similar.

Improve Documentation

Whenever behaviour becomes clearer, documentation should evolve alongside it.

Examples include:

- package comments
- exported API documentation
- ADRs
- architecture specifications
- README updates

Code and documentation should describe the same system.

Documentation drift is technical debt.

Improve Tests

Every modification should consider testing.

Examples:

- additional edge case
- improved assertions
- removal of duplicate tests
- clearer fixtures
- improved test names

Tests are first-class engineering artefacts.

They deserve the same attention as production code.

Respect Existing Style

Improvement should move software towards consistency.

Not personal preference.

Follow:

- existing repository conventions
- Go conventions
- MEG standards

The objective is a coherent codebase.

Not individual expression.

Know When To Stop

The Boy Scout Rule encourages improvement.

It does not encourage perfectionism.

If a refactoring begins expanding beyond the original scope, stop.

Consider:

- separate pull request
- ADR
- future engineering task

Small improvements are sustainable because they remain small.

Engineering Ownership

Every engineer shares responsibility for the quality of the codebase.

Ownership does not end at package boundaries.

If improvement is obvious and safe, it should be made.

The codebase belongs to the team.

Not individual authors.

Examples

Good

```
Implement feature

↓

Rename confusing variable

↓

Add missing test

↓

Improve package comment
```

Better

```
Implement feature

↓

Simplify control flow

↓

Remove duplication

↓

Improve logging

↓

Update documentation

↓

Add benchmark
```

Poor

```
Implement feature

↓

Ignore obvious problems

↓

Leave dead code

↓

Introduce more duplication
```

Technical debt rarely arrives dramatically.

It accumulates quietly.

Engineering Mindset

The Boy Scout Rule encourages engineers to ask one simple question before every commit.

> ***"Is this repository better than it was before I started?"***

The improvement does not need to be large.

It simply needs to be genuine.

Mosaic Guidelines

Within Mosaic:

- Every pull request SHOULD improve the surrounding code where practical.
- Naming SHOULD become clearer over time.

- Dead code SHOULD be removed.
- Documentation SHOULD evolve alongside implementation.
- Tests SHOULD improve continuously.
- Refactoring SHOULD remain incremental.
- Engineers SHOULD optimise for the long-term health of the repository.
- Large rewrites SHOULD be avoided unless architecturally justified.

Relationship to the MEG

The Boy Scout Rule is not another engineering principle.

It is the mechanism through which every other principle remains effective over time.

Without continuous improvement:

- architecture drifts
- consistency erodes
- complexity accumulates
- technical debt compounds

With continuous improvement:

- software remains understandable
- standards remain enforceable
- onboarding becomes easier
- refactoring becomes cheaper

The Boy Scout Rule transforms engineering quality from a periodic activity into a daily habit.

Summary

Every engineer leaves a mark on a codebase.

Within Mosaic, that mark should always be positive.

No change is too small to improve something.

The cumulative effect of thousands of small improvements is a platform that remains maintainable, understandable and enjoyable to work on for many years.

That is the true objective of the Boy Scout Rule.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`15-code-review-standards.md`

Next File

`17-adrs.md`

Architectural Decision Records (ADRs)

> *Every significant architectural decision should be recorded once, explained clearly and referenced often.*

Purpose

Architecture is a sequence of decisions.

Over time, engineers inevitably ask:

- Why was this technology chosen?
- Why is the package structured this way?
- Why don't we use a dependency injection framework?
- Why are events preferred over direct service calls?

Without documentation, these discussions repeat.

Architectural Decision Records (ADRs) capture the reasoning behind significant engineering decisions so future contributors understand **why** the software exists in its current form.

Philosophy

Within Mosaic:

> ***Architecture should explain itself.***

The codebase explains *what* the software does.

ADRs explain *why* it does it that way.

Every major engineering decision should remain understandable years after it was made.

What Is An ADR?

An Architectural Decision Record is a short document describing:

- the problem
- the available options
- the chosen solution
- the reasoning
- the consequences

An ADR records a decision at a point in time.

It is **not** intended to become a design document or implementation guide.

Why ADRs Matter

Software outlives conversations.

Months later, engineers rarely remember:

- trade-offs
- rejected alternatives
- historical constraints
- business drivers

Without ADRs, teams often repeat previous discussions because the original reasoning has been lost.

ADRs preserve architectural knowledge.

When To Write An ADR

An ADR SHOULD be created whenever a decision:

- significantly influences architecture
- changes engineering standards
- introduces new infrastructure
- affects multiple repositories
- changes dependency direction
- introduces a new pattern
- removes an existing pattern
- establishes a long-term convention

Small implementation decisions do not require ADRs.

Architectural decisions do.

Examples

Examples of decisions that SHOULD have ADRs include:

- Choosing Go as the implementation language.
- Choosing PostgreSQL as the primary database.
- Introducing DuckDB for analytics.
- Event-driven architecture.
- Repository package structure.
- Extension SDK architecture.
- Refraction rendering engine.

- Blob storage abstraction.
- Background worker model.
- Docker orchestration strategy.

These decisions affect the platform for years.

Their rationale should therefore remain discoverable.

What Should Not Become An ADR

The following generally SHOULD NOT become ADRs.

- Bug fixes
- Refactoring
- Naming changes
- Package moves
- Small implementation details
- Formatting standards

If reversing the decision would not significantly affect architecture, an ADR is probably unnecessary.

ADR Lifecycle

Every ADR follows the same lifecycle.

```
Proposed

↓

Accepted

↓

Implemented

↓

Superseded (optional)

↓

Archived
```

Most ADRs will remain "Accepted" for many years.

Superseded ADRs remain valuable historical references.

They should never be deleted.

ADR Structure

Every ADR SHOULD follow a consistent structure.

```
Title
↓
Status
↓
Context
↓
Problem
↓
Options Considered
↓
Decision
↓
Consequences
↓
Related Documents
```

Consistency makes ADRs significantly easier to navigate.

Context

The Context section explains:

- the current situation
- existing constraints
- relevant background
- why the decision became necessary

Readers unfamiliar with the project should understand why the discussion exists.

Problem Statement

The problem statement should be objective.

Good:

> *The platform requires a high-performance metadata store capable of analytical queries.*

Poor:

> *We wanted to try DuckDB.*

The problem should exist independently of the proposed solution.

Options

Every significant alternative SHOULD be documented.

Example.

`PostgreSQL`

`DuckDB`

`SQLite`

Each option should include:

- benefits
- drawbacks
- implementation impact

Rejected alternatives are often as valuable as accepted ones.

Decision

The decision section answers one question.

> ***What was chosen?***

The decision should be concise.

The reasoning belongs elsewhere.

Consequences

Every architectural decision introduces consequences.

Examples include:

Positive:

- improved performance
- reduced coupling
- easier testing

Negative:

- additional complexity
- operational cost
- migration effort

Trade-offs should be documented honestly.

No architecture is free.

Superseding Decisions

Architecture evolves.

When replacing an existing ADR:

- create a new ADR
- reference the previous ADR
- mark the old ADR as superseded

Do not modify history.

Record evolution instead.

Cross References

Every ADR SHOULD reference related specifications where appropriate.

Examples:

```
MEG-001

↓

MDS-006

↓

ADR-012
```

Architecture should form a navigable knowledge graph.

Not isolated documents.

Repository Structure

Recommended layout.

```
architecture/

    adrs/

        ADR-001-go-language.md

        ADR-002-event-driven.md

        ADR-003-postgresql.md

        ADR-004-duckdb.md

        ADR-005-extension-sdk.md
```

ADRs should remain close to the architectural documentation.

Not hidden inside implementation repositories.

Writing Guidelines

An ADR should be:

- concise
- objective
- factual
- timeless

Avoid:

- implementation details
- emotional language
- unnecessary technical depth

The objective is to explain a decision.

Not reproduce the implementation.

Review

Architectural decisions SHOULD be reviewed before acceptance.

Reviews should consider:

- alternatives
- trade-offs
- long-term impact
- alignment with existing architecture

Accepted ADRs become part of Mosaic's engineering history.

Mosaic Guidelines

Within Mosaic:

- Significant architectural decisions SHOULD have ADRs.
- ADRs MUST explain why, not merely what.
- Alternatives SHOULD be documented.
- Consequences MUST be acknowledged.
- Historical ADRs MUST NOT be deleted.
- Superseded ADRs SHOULD remain discoverable.
- Architecture specifications SHOULD reference relevant ADRs where appropriate.

Relationship to the MEG

The MEG defines engineering standards.

ADRs explain why those standards exist.

Whenever an engineering standard changes significantly, the corresponding ADR should explain the reasoning behind the change.

Together they provide:

```
ADR

↓

Reasoning

↓

MEG

↓

Engineering Standard

↓

Implementation
```

This relationship ensures that architectural intent remains preserved long after individual contributors have moved on.

Summary

Software changes.

Architecture evolves.

People leave projects.

ADRs ensure that engineering knowledge remains.

Future contributors should never need to rediscover important architectural decisions through trial and error.

Instead, they should be able to read the decision, understand the reasoning and continue building upon it with confidence.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`16-boy-scout-rule.md`

Next File

`18-contributor-guidance.md`

Contributor Guidance

> *Consistency is the responsibility of every contributor. The goal is not simply to write working code, but to write code that belongs.*

Purpose

This document provides practical guidance for engineers contributing to the Mosaic ecosystem.

The previous chapters establish engineering standards.

This chapter explains how contributors should apply those standards during day-to-day development.

Every contribution, regardless of size, should leave the codebase more understandable, more maintainable and more consistent than before.

Philosophy

Within Mosaic:

> ***Contributors do not add code. They improve the platform.***

Every commit should contribute towards one or more of the following:

- new functionality
- improved readability
- improved maintainability
- reduced complexity
- improved documentation
- improved reliability
- improved observability

Features alone are not progress.

Quality matters equally.

Before Writing Code

Before beginning implementation, every contributor SHOULD understand:

- the problem being solved
- the architectural boundaries involved
- the affected packages
- existing engineering standards
- relevant ADRs

- related specifications

Code should be the final step.

Understanding comes first.

Before Creating A Package

Ask:

- Does this responsibility already exist?
- Can the existing package evolve naturally?
- Would a new package improve clarity?
- Does this introduce unnecessary coupling?

New packages should emerge from architectural necessity.

Not personal preference.

Before Creating An Interface

Ask:

- Is there more than one consumer?
- Does this reduce coupling?
- Am I solving today's problem?
- Would a concrete type be simpler?

If uncertainty exists, begin with a concrete implementation.

Interfaces are easier to introduce later than remove.

Before Introducing Concurrency

Ask:

- What problem does concurrency solve?
- Who owns this goroutine?
- How is cancellation handled?
- How are errors propagated?
- What happens during shutdown?

Concurrency without ownership is prohibited.

Before Optimising

Ask:

- Has the application been profiled?
- Is this actually a bottleneck?
- Can measurements demonstrate improvement?
- Does readability suffer?

Optimisation without evidence should not proceed.

Before Merging

Every contributor SHOULD complete the following checklist.

Architecture

- Responsibilities remain clear.
- Dependencies flow correctly.
- Package ownership remains consistent.
- No unnecessary abstraction introduced.

Correctness

- Errors handled.
- Edge cases considered.
- Context propagated.
- Resources cleaned up.

Testing

- New behaviour tested.
- Existing tests pass.
- Race detector considered.
- Benchmarks added where appropriate.

Documentation

- Exported APIs documented.
- Package comments updated where necessary.
- ADR referenced if applicable.

- Architecture documentation updated if required.

Go expects exported packages and exported identifiers to have documentation comments. Documentation is part of the public API, not an optional extra.
[oai_citation:0‡Go](https://go.dev/doc/comment?utm_source=chatgpt.com)

Quality

- Dead code removed.
- Naming improved.
- Duplication reduced.
- Complexity simplified where practical.

Every pull request should leave the surrounding code better than before.

Commit Philosophy

Commits SHOULD represent one logical change.

Good commits are:

- focused
- reviewable
- reversible
- understandable

Avoid commits that simultaneously:

- refactor architecture
- introduce new features
- rename packages
- reformat unrelated files

Small commits produce better reviews.

Pull Requests

A pull request should answer three questions.

What changed?

Describe the behaviour.

Not merely the files.

Why?

Explain the motivation.

Reference:

- issue
- ADR
- architecture specification

where appropriate.

How was it validated?

Examples include:

- unit tests
- integration tests
- benchmarks
- manual verification

Confidence should be demonstrated.

Not assumed.

Documentation Expectations

Documentation is considered production code.

Every contributor is responsible for maintaining it.

When behaviour changes:

Update documentation.

Do not defer documentation updates.

Documentation that disagrees with implementation is a defect.

Naming

Engineers SHOULD spend time choosing names.

Good names reduce:

- comments
- documentation
- onboarding effort
- bugs

Poor names create confusion that persists for years.

Rename aggressively when understanding improves.

Ask Questions

No contributor is expected to know everything.

When uncertainty exists:

- consult the MEG
- consult relevant ADRs
- discuss architectural trade-offs
- ask for review early

It is cheaper to ask questions than rewrite software.

Learning The Codebase

New contributors SHOULD prioritise understanding before modification.

Recommended order:

```
Architecture Specifications
↓
ADRs
↓
Package Documentation
↓
Public APIs
↓
Implementation
↓
Tests
```

Understanding architecture first reduces accidental complexity later.

Open Source Contributions

External contributors are expected to follow the same engineering standards as internal contributors.

Review criteria remain identical.

Code quality should never depend upon who authored the change.

Consistency matters more than authorship.

Engineering Culture

Every contributor should strive to:

- leave constructive review comments
- share architectural knowledge
- improve documentation
- simplify existing code
- mentor newer contributors
- question unnecessary complexity

Healthy engineering culture produces healthy software.

Things To Avoid

Avoid contributing code that:

- introduces hidden dependencies
- bypasses architecture
- duplicates existing behaviour
- ignores engineering standards
- increases complexity unnecessarily
- solves hypothetical future problems

The simplest correct solution is usually preferred.

Contributor Checklist

Before requesting review, confirm:

- [ ] The implementation follows the MEG.
- [ ] Package ownership remains clear.
- [ ] Dependencies are explicit.
- [ ] Errors are handled correctly.
- [ ] Context is propagated.
- [ ] Tests pass.
- [ ] Documentation is updated.
- [ ] Dead code removed.
- [ ] Naming is clear.
- [ ] The repository is better than before.

Relationship to the MEG

This document does not introduce additional engineering rules.

Instead, it explains how contributors should apply the standards established throughout MEG-001 during everyday development.

Engineering standards only improve software when contributors consistently follow them.

Summary

Software quality is a shared responsibility.

Every contributor influences:

- architecture
- maintainability
- readability
- engineering culture

Within Mosaic, contributions are measured not only by the features they introduce, but by the lasting improvements they make to the platform.

The best contribution is one that future contributors barely notice because it feels like it has always belonged.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`17-adrs.md`

Next File

`glossary.md`

Glossary

> *Engineering terminology should have one meaning throughout the Mosaic ecosystem.*

Purpose

This glossary defines the terminology used throughout the Mosaic Engineering Guidelines (MEG).

Where a term has a specific architectural meaning within Mosaic, that definition takes precedence over colloquial usage.

The purpose of this glossary is to ensure that engineers discuss architecture using a shared vocabulary.

A

Abstraction

A simplified representation of behaviour that intentionally hides implementation details.

Within Mosaic, abstractions exist to reduce coupling and improve maintainability.

Abstractions MUST justify their existence by simplifying the software.

API

Application Programming Interface.

The public contract exposed by a package, service or application.

Within Mosaic, exported identifiers form part of a package's API.

Architecture

The high-level organisation of software components, responsibilities and dependencies.

Architecture answers:

> *How is the system organised?*

Not:

> *How is a particular function implemented?*

Architectural Boundary

A clear separation between responsibilities.

Examples include:

- Transport
- Domain

- Infrastructure
- Storage

Boundaries reduce coupling and improve maintainability.

B

Behaviour

The observable actions provided by a component.

Go interfaces describe behaviour rather than implementation.

Business Logic

Rules that implement domain-specific behaviour.

Business logic SHOULD remain independent of:

- HTTP
- databases
- transport protocols
- infrastructure

Boundary

A location where responsibility changes.

Examples include:

- HTTP → Service
- Service → Repository
- Repository → Database

Boundaries often translate data, errors or models.

C

Cancellation

The act of terminating ongoing work through `context.Context`.

All long-running Mosaic operations are expected to honour cancellation requests.

Cohesion

The degree to which responsibilities within a package naturally belong together.

High cohesion is desirable.

Composition

Building complex behaviour by combining smaller components.

Composition is the preferred architectural mechanism within Mosaic.

Composition Root

The location where application dependencies are constructed.

Typically:

`cmd/server/main.go`

Concurrency

Structuring software so multiple independent tasks can make progress.

Concurrency improves responsiveness.

It does not necessarily imply parallel execution.

Coupling

The degree to which one component depends upon another.

Lower coupling generally produces more maintainable software.

D

Dependency

A component required by another component.

Dependencies should be:

- explicit
- constructor-injected
- visible

Domain

The business concepts represented by the software.

Examples include:

- Library
- Playback

- Metadata
- User

The domain should remain independent of infrastructure.

E

Embedding

A Go language feature allowing one type to promote another type's methods.

Embedding is not inheritance.

Within Mosaic, embedding is used sparingly.

Error Boundary

A point where errors are translated into a more meaningful representation.

Example:

```
SQL
↓
Repository
↓
Domain Error
↓
HTTP Response
```

G

Goroutine

A lightweight concurrent execution managed by the Go runtime.

Every goroutine within Mosaic must have:

- an owner
- cancellation
- lifecycle management

I

Infrastructure

Components responsible for interacting with external systems.

Examples include:

- databases
- caches
- message brokers
- blob storage

Infrastructure should remain isolated from business logic.

Interface

A behavioural contract describing capabilities.

Interfaces belong to consumers.

They do not describe implementation.

M

Middleware

Behaviour executed before or after another component.

Examples include:

- authentication
- logging
- metrics
- tracing

Each middleware should own one responsibility.

Mutation

Modification of existing state.

Mutable shared state requires explicit synchronisation.

O

Observability

The ability to understand system behaviour through:

- logs
- metrics
- traces

- health checks

Every production service should be observable.

P

Package

The primary architectural unit within Go.

Packages own responsibilities.

Types implement behaviour.

Polymorphism

The ability for multiple implementations to satisfy the same interface.

Within Go, polymorphism is achieved through interfaces rather than inheritance.

Public API

The exported behaviour exposed by a package.

Public APIs should remain small and stable.

R

Refactoring

Improving internal implementation without changing observable behaviour.

Refactoring should improve:

- readability
- maintainability
- simplicity

Repository

A component responsible for persistence.

Repositories should not contain business logic.

S

Service

A component responsible for implementing business behaviour.

Services coordinate collaborators.

They do not own infrastructure.

Single Responsibility

The principle that a component should have one reason to change.

This principle applies equally to:

- packages
- structs
- functions
- services

State

The current condition of a component.

Mutable state should remain clearly owned.

Strategy

An interchangeable implementation satisfying a common behaviour.

Strategies naturally emerge from Go interfaces.

T

Technical Debt

The long-term cost introduced by suboptimal engineering decisions.

Technical debt should be reduced continuously.

Not accumulated deliberately.

Transport

The mechanism through which requests enter or leave an application.

Examples include:

- HTTP

- CLI
- WebSocket
- gRPC

Transport should remain separate from business logic.

U

Utility Package

A generic package lacking a clearly defined responsibility.

Examples include:

`utils`

`common`

`helpers`

Utility packages are prohibited within Mosaic.

W

Worker Pool

A bounded collection of goroutines processing queued work.

Worker pools provide predictable concurrency and resource usage.

Common Acronyms

Acronym	Meaning
ADR	Architectural Decision Record
API	Application Programming Interface
CI	Continuous Integration
GC	Garbage Collector
HTTP	Hypertext Transfer Protocol
MEG	Mosaic Engineering Guidelines
MDL	Mosaic Design Language
MDS	Mosaic Design Specifications
SDK	Software Development Kit
SQL	Structured Query Language
SWR	Stale-While-Revalidate

Relationship to the MEG

This glossary supports every document within MEG-001.

Definitions should remain consistent across:

- engineering standards
- ADRs
- architecture specifications
- contributor documentation

Where terminology evolves, this glossary SHOULD be updated before introducing new definitions elsewhere.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`18-contributor-guidance.md`

Next File

`references.md`

References

> *Engineering standards should be grounded in established knowledge rather than individual opinion.*

Purpose

The Mosaic Engineering Guidelines (MEG) build upon decades of software engineering experience.

This document records the primary references that informed the standards defined throughout MEG-001.

These references should be considered recommended reading for contributors wishing to deepen their understanding of Go and software architecture.

The MEG is intentionally opinionated.

However, those opinions are derived from established engineering principles rather than invented in isolation.

Primary Go References

Effective Go

Publisher

The Go Team

Purpose

The canonical guide to writing idiomatic Go.

Topics

- Language idioms
- Package design
- Interfaces
- Concurrency
- Error handling
- Naming
- Documentation

URL

https://go.dev/doc/effective_go

Go Code Review Comments

Publisher

The Go Team

Purpose

Practical engineering guidance used by Go maintainers during code review.

Topics

- Package naming
- Error handling
- Context usage
- Interfaces
- Receiver design
- Comments
- Testing
- Style

URL

https://go.dev/wiki/CodeReviewComments

Go Blog

Publisher

The Go Team

Purpose

Articles explaining language design, performance, concurrency and architecture.

Recommended topics include:

- Context
- Pipelines
- Profiling
- Error handling
- Memory
- Synchronisation

URL

https://go.dev/blog

Go Package Documentation

Publisher

The Go Team

Purpose

Authoritative documentation for the Go standard library.

Examples include:

- context
- errors
- sync
- net/http
- slog
- testing

URL

https://pkg.go.dev

Software Engineering References

Clean Architecture

Author

Robert C. Martin

Topics

- Dependency direction
- Architectural boundaries
- Separation of concerns

Clean Code

Author

Robert C. Martin

Topics

- Readability
- Naming
- Functions
- Refactoring

While several examples are Java-centric, many of the underlying engineering principles remain valuable.

Individual implementation patterns should always be adapted to idiomatic Go.

Domain-Driven Design

Author

Eric Evans

Topics

- Domain modelling
- Ubiquitous language
- Bounded contexts
- Aggregates

The strategic concepts remain highly relevant.

Implementation techniques should follow Go idioms rather than object-oriented conventions.

Refactoring

Author

Martin Fowler

Topics

- Incremental improvement
- Code smells
- Continuous evolution
- Technical debt reduction

This work strongly aligns with the Boy Scout Rule adopted throughout the MEG.

The Pragmatic Programmer

Authors

Andrew Hunt

David Thomas

Topics

- Software craftsmanship
- Continuous improvement
- Engineering discipline

Many of the engineering attitudes reflected within the MEG originate from this work.

Designing Data-Intensive Applications

Author

Martin Kleppmann

Topics

- Distributed systems
- Data consistency
- Event-driven systems

- Storage architecture
- Scalability

Strongly recommended reading for engineers working on distributed Mosaic services.

Architecture References

The following architectural concepts influence the Mosaic ecosystem.

- Event-Driven Architecture
- Hexagonal Architecture
- Ports and Adapters
- CQRS (where appropriate)
- Stale-While-Revalidate
- Repository Pattern
- Worker Pool Pattern
- Pipeline Pattern

Mosaic intentionally adopts concepts rather than frameworks.

Architectural ideas should always be evaluated through the lens of simplicity and maintainability.

Concurrency References

Recommended reading.

Go Concurrency Patterns

The Go Team

Topics include:

- Pipelines
- Cancellation
- Fan-out
- Fan-in

https://go.dev/blog/pipelines

Share Memory By Communicating

The Go Team

Introduces Go's concurrency philosophy.

https://go.dev/blog/share-memory-by-communicating

The Context Package

The Go Team

Introduces structured cancellation throughout Go applications.

https://go.dev/blog/context

Testing References

Recommended reading.

- Go Testing Package Documentation
- Go Fuzz Testing
- Go Race Detector
- Table Driven Tests

Useful URLs:

https://pkg.go.dev/testing

https://go.dev/doc/articles/race_detector

https://go.dev/wiki/TableDrivenTests

Performance References

Recommended reading.

- Profiling Go Programs
- Escape Analysis
- Go Memory Model
- Performance Wiki

Useful URLs:

https://go.dev/blog/profiling-go-programs

https://go.dev/ref/mem

https://go.dev/wiki/Performance

Logging & Observability

Recommended references.

- slog Package Documentation
- OpenTelemetry Specification
- Prometheus Best Practices

Observability should be considered an architectural concern rather than an operational afterthought.

Internal References

The following Mosaic specifications complement MEG-001.

Mosaic Design Language

- MDL-001 Vision
- MDL-002 Principles
- MDL-003 Mental Model
- MDL-004 Interaction Model
- MDL-005 Composition Model

Mosaic Design Specifications

- MDS-001 Design Token Architecture
- MDS-002 Colour System
- MDS-003 Material System
- MDS-004 Typography System
- MDS-005 Motion System
- MDS-006 Composition Engine
- MDS-007 Tile Framework
- MDS-008 Component Library

Future Mosaic Architecture Specifications

Examples include:

- Event Architecture
- Storage Architecture
- Extension SDK
- Authentication
- Metadata Platform
- Runtime Platform
- Observability
- Deployment Architecture

Together these specifications define the complete architectural blueprint for the Mosaic ecosystem.

Keeping References Current

Engineering practices evolve.

Languages evolve.

Tooling evolves.

This document SHOULD be reviewed periodically to ensure referenced material remains:

- relevant
- accessible
- authoritative

Where better references become available, they should replace older material while preserving the engineering philosophy established throughout the MEG.

Closing Statement

The Mosaic Engineering Guidelines do not exist to replace established software engineering knowledge.

They exist to curate, refine and adapt that knowledge into a consistent set of standards for the Mosaic platform.

Contributors are encouraged to understand not only **what** these standards require, but also **why** they exist.

Engineering judgement will always remain more valuable than memorising rules.

Review Status

Status

Draft

Owner

Lead Software Architect

Previous File

`glossary.md`

Next File

End of Specification